

Chapter 1

INTRODUCTION

In the modern digital era, business organizations rely heavily on information technology systems and network infrastructures to perform their daily operations. While this dependence improves efficiency and productivity, it also exposes organizations to various cybersecurity threats such as unauthorized access, data breaches, malware attacks, and insider threats. The increasing use of remote work environments and cloud-based services has further expanded the attack surface, making it more challenging to ensure the security of organizational assets.

To address these challenges, organizations require effective mechanisms to monitor, detect, and respond to security incidents in real time. Traditional security approaches, which rely on isolated tools and manual analysis, are no longer sufficient to handle the large volume of security events generated across multiple systems. This has led to the adoption of Security Information and Event Management (SIEM) systems, which provide a centralized platform for collecting, analysing, and managing security-related data.

A SIEM system aggregates logs and events from various sources such as servers, endpoints, network devices, and applications. It processes and correlates this data to identify patterns that may indicate potential security threats. By enabling real-time monitoring and automated alert generation, SIEM systems help organizations detect suspicious activities at an early stage and respond promptly. Additionally, SIEM solutions support Security Operations Centre (SOC) activities by assisting analysts in monitoring, investigating, and managing security incidents.

In addition to providing centralized monitoring, SIEM systems play a crucial role in enhancing the overall cybersecurity posture of an organization. With the increasing volume of data generated by modern IT infrastructures, it becomes essential to have an automated system capable of efficiently handling, processing, and analysing large amounts of security-related information. SIEM systems are specifically designed to address this challenge by integrating multiple security functions into a single unified platform.

One of the key advantages of SIEM systems is their ability to perform **real-time log analysis**. Logs generated from different devices such as servers, firewalls, routers, and endpoints are continuously collected and analysed. This enables organizations to identify unusual patterns or anomalies that may indicate potential threats. For example, repeated failed login attempts from a single IP address may indicate a brute-force attack, while unusual access patterns could suggest insider threats or compromised accounts.

Furthermore, SIEM systems support **event correlation**, which is a critical feature for detecting complex cyber-attacks. Instead of analysing individual events in isolation, SIEM correlates multiple events across different systems to identify suspicious patterns. This helps in detecting advanced threats that may otherwise go unnoticed when using traditional security tools. By correlating data from multiple sources, the system can provide a more accurate and comprehensive view of the organization's security status.

Another important aspect of SIEM systems is **alert generation and notification**. When a potential threat is detected, the system generates alerts to notify security analysts. These alerts can be prioritized based on severity, allowing analysts to focus on critical issues first. This significantly reduces response time and helps in preventing potential security breaches. Automated alerting also ensures that no important event is missed, even in environments with high volumes of data.

SIEM systems also provide **dashboard visualization**, which allows users to monitor security events through graphical representations such as charts, graphs, and reports. These dashboards offer a clear and concise overview of system activities, making it easier for security teams to understand and analyse data. Visualization tools help in identifying trends, patterns, and anomalies, thereby supporting better decision-making.

In addition to monitoring and detection, SIEM systems play a vital role in **incident response and management**. Once a threat is detected, the system provides detailed information about the event, including the source, time, and type of attack. This enables security analysts to investigate the incident thoroughly and take appropriate action. SIEM systems often integrate with incident response tools to automate certain actions, such as blocking malicious IP addresses or isolating affected systems.

The implementation of a SIEM system also supports the functioning of a **Security Operations Centre (SOC)**. A SOC is responsible for continuously monitoring and analysing security events within an organization. SIEM acts as the backbone of SOC operations by providing the necessary tools and data required for effective monitoring and analysis. Tier 1 analysts are responsible for monitoring alerts and performing initial investigations, while Tier 2 analysts conduct deeper analysis and handle complex incidents.

Moreover, SIEM systems contribute to **regulatory compliance and reporting**. Many organizations are required to comply with security standards and regulations that mandate proper monitoring and logging of system activities. SIEM systems help in maintaining detailed logs and generating reports that can be used for auditing and compliance purposes. This ensures that organizations meet legal and regulatory requirements while maintaining a secure environment.

Despite their advantages, implementing SIEM systems also presents certain challenges. These include handling large volumes of data, configuring appropriate rules, and minimizing false positives. However, with proper configuration and tuning, these challenges can be effectively managed. The use of open-source SIEM tools such as ELK Stack and Wazuh makes it possible to implement cost-effective solutions without compromising on functionality.

In this project, the SIEM system is designed and implemented using open-source technologies to demonstrate how organizations can achieve effective security monitoring with minimal cost. The system collects logs from multiple client systems, processes them using log management tools, and stores them in a centralized database. It then applies rule-based detection mechanisms to identify suspicious activities and generates alerts accordingly.

The project also demonstrates how security analysts interact with the system to monitor events and respond to incidents. Through the use of dashboards, analysts can visualize system activities and gain insights into potential threats. This practical implementation highlights the importance of SIEM systems in modern cybersecurity and provides a foundation for further research and development in this field.

In conclusion, SIEM systems have become an essential component of modern cybersecurity infrastructure. They provide a comprehensive solution for monitoring, detecting, and responding to security threats in real time. By integrating multiple security functions into a single platform, SIEM systems enhance visibility, improve efficiency, and enable organizations

to proactively manage security risks. As cyber threats continue to evolve, the role of SIEM systems will become even more critical in ensuring the safety and integrity of organizational assets. In addition to real-time monitoring and threat detection, SIEM systems also play a significant role in improving the overall efficiency of security management within an organization. By automating log collection and analysis, the system reduces the need for manual monitoring, allowing security teams to focus on critical tasks. This not only saves time but also minimizes human errors in identifying potential threats.

Another important benefit of SIEM systems is their ability to provide historical analysis. Stored logs can be reviewed to investigate past incidents, identify recurring threats, and improve future security strategies. This helps organizations strengthen their defence mechanisms and prepare for evolving cyber threats.

Furthermore, SIEM systems support scalability, enabling organizations to expand their infrastructure without compromising security. As new devices and applications are added, the system can easily integrate and monitor them. This flexibility makes SIEM an essential solution for modern organizations aiming to maintain strong and reliable cybersecurity practices.

Moreover, SIEM systems enhance the ability of organizations to respond quickly to security incidents by providing detailed insights into system activities. When a security event occurs, the system not only generates alerts but also provides contextual information such as the source of the attack, affected systems, and the timeline of events. This helps security analysts perform efficient investigation and take appropriate actions to mitigate risks.

Another important advantage of SIEM systems is their support for continuous monitoring. The system operates 24/7, ensuring that all activities are tracked without interruption. This is especially important in today's environment where cyber-attacks can occur at any time. Continuous monitoring ensures that threats are detected early and handled before they cause significant damage.

Additionally, SIEM systems help in improving organizational decision-making by providing accurate and real-time security data. Managers and security teams can use this information to develop better security policies and strategies, thereby strengthening the overall cybersecurity framework of the organization.

Chapter 2

PROJECT DESCRIPTION

2.1 PROBLEM DEFINITION

The Security Information and Event Management (SIEM) system is a cybersecurity solution used to collect, monitor, analyse, and manage security events from different systems within a business organization. It provides a centralized platform where logs and activities from servers, computers, and network devices are gathered and analysed in real time.

The main purpose of this project is to design and implement a SIEM system that helps organizations detect suspicious activities, generate alerts, and respond to security incidents effectively. The system improves visibility into security events and supports Security Operations Centre (SOC) activities by assisting analysts in monitoring and investigating threats.

In simple terms, this project focuses on building a system that:

1. Collects logs from multiple systems
2. Monitors activities continuously
3. Detects security threats
4. Generates alerts
5. Helps in analysing and responding to incidents

2.2 PROJECT DETAILS

The project focuses on the design and implementation of a Security Information and Event Management (SIEM) system to monitor and manage security events in a business organization. The system is developed using open-source tools and follows a centralized architecture to collect and analyse security-related data from multiple sources. In this project, logs are collected from different systems such as client machines, servers, and network devices using log collection agents. These logs include information related to user activities, login attempts, system processes, and network events. The collected data is then sent to a central SIEM server where it is processed, stored, and analysed.

Design And Implementation of Security Information and Event Management (SIEM) System for Monitoring Security Events in A Business Organization

The SIEM system performs real-time monitoring and event correlation to identify suspicious activities and potential security threats. It uses predefined rules to detect abnormal behaviour such as multiple failed login attempts, unauthorized access, and unusual system activity. When such events are detected, the system generates alerts to notify security analysts.

The project also includes the implementation of dashboard visualization, which provides graphical representation of logs, alerts, and system activities. This helps in understanding security patterns and improves decision-making. Additionally, the system supports Security Operations Centre (SOC) functions by simulating the roles of Tier 1 and Tier 2 analysts. Tier 1 analysts monitor alerts and perform initial analysis, while Tier 2 analysts investigate and validate security incidents.

Overall, the project demonstrates how a SIEM system can be used to enhance security monitoring, improve incident response, and provide better visibility into organizational activities. It provides a practical approach to implementing cybersecurity solutions in a business environment.

Furthermore, the proposed SIEM system emphasizes scalability and flexibility to adapt to the growing needs of an organization. As businesses expand their infrastructure by adding new devices, applications, and users, the system can seamlessly integrate these components and continue to monitor security events without performance degradation. This ensures that the organization maintains a consistent level of security across all its operations.

The system also enhances data management by maintaining a centralized repository of logs, which can be used for both real-time analysis and historical investigation. This capability allows security teams to track past incidents, identify recurring threats, and improve future security strategies. In addition, the system supports automation in alert handling, reducing manual effort and enabling faster response to critical threats.

By combining real-time monitoring, efficient data handling, and automated alert mechanisms, the SIEM system provides a reliable and effective solution for modern cybersecurity challenges. This makes it highly suitable for organizations aiming to strengthen their security infrastructure and ensure continuous protection against evolving cyber threats.

Chapter 3

COMPUTATIONAL ENVIRONMENT

3.1 SOFTWARE REQUIRMENTS

The software requirements document is the specification of the system. It includes both the definition and specification of requirements. It describes what the system should perform rather than how it is implemented. The software requirements provide a foundation for developing the system, estimating cost, planning tasks, and tracking project progress throughout the development lifecycle.

Software Requirements Specification

Application Platform	:	Python 3.0 & above
Operating System	:	Windows 10 / Ubuntu (for SIEM Server)
Database used	:	Elasticsearch
Log Collection Tools	:	Wazuh
Alerting System	:	Wazuh
Virtualization	:	VirtualBox

3.2 HARDWARE REQUIRMENTS

The hardware requirements may serve as the basis for a contract for the implementation of the system and should therefore be a complete and consistent specification of the whole system. They are used by software engineers as the starting point for the system design. It shows what the system does and not how it should be implemented. Ambiguities or omissions in these requirements can lead to significant cost overruns and delays during the project lifecycle.

RAM	:	8GB & Above
Processor	:	Intel i5 & Above
Hard Disk	:	5 GB & Above

3.3 FUNCTIONAL REQUIRMENTS

The Security Information and Event Management (SIEM) system is designed to perform several essential functions for effective monitoring and management of security events within a business organization. Initially, the system allows the collection of log data from multiple sources such as client systems, servers, and network devices. These logs include information related to user activities, login attempts, system processes, and network events. The collected data is securely transmitted and stored in a centralized repository.

Once the data is collected, it undergoes a processing stage where logs are parsed, normalized, and structured to ensure consistency and accuracy in analysis. This preprocessing step enables efficient handling of large volumes of data and improves the effectiveness of threat detection. The system continuously monitors incoming logs in real time to identify abnormal or suspicious activities.

The SIEM system integrates rule-based detection mechanisms to analyse security events. These rules help in identifying patterns such as multiple failed login attempts, unauthorized access, and unusual system behaviour. Based on these patterns, the system generates alerts to notify security analysts about potential threats. The system also supports real-time monitoring through dashboards, allowing analysts to visualize logs, alerts, and system activities.

Additionally, the system supports Security Operations Centre (SOC) activities by enabling Tier 1 and Tier 2 analysts to monitor, investigate, and respond to security incidents. Tier 1 analysts focus on alert monitoring and initial analysis, while Tier 2 analysts perform deeper investigation and validation of incidents

3.4 SOFTWARE FEATURES

SIEM System Wazuh:

The Security Information and Event Management (SIEM) system is used for monitoring, analysing, and managing security events within a business organization. It provides a centralized platform for collecting logs from multiple systems and detecting suspicious activities in real time. The system integrates various tools to ensure efficient security monitoring and incident response. The SIEM system is designed to be scalable, flexible, and easy to use. It supports real-time data processing, alert generation, and visualization through dashboards. It helps organizations improve their security posture by identifying threats early

and responding quickly. Some of the main features of the SIEM system are as follows: Centralized logging, Real-time monitoring, Event correlation, Alert generation, Dashboard visualization, and Incident analysis. It is widely used in cybersecurity for threat detection, log analysis, and security operations.

Core Tools Used:

Elasticsearch – Used for storing and indexing logs

Logstash – Used for collecting and processing logs

Kibana – Used for dashboards and visualization

Wazuh – Used for security monitoring and alerting (optional integrated SIEM)

Advantages:

i. Centralized Monitoring:

Collects logs from multiple systems into one place, making it easier to monitor and manage security events.

ii. Real-Time Detection:

Detects suspicious activities instantly and helps in quick response to threats.

iii. Improved Security Visibility:

Provides complete visibility of system activities through dashboards and logs.

iv. Scalability:

Can be expanded easily by adding more systems and log sources.

v. Open-Source and Cost-Effective:

Uses free tools, reducing implementation cost for organizations.

vi. Efficient Incident Response:

Generates alerts and helps security teams analyze and respond to incidents quickly.

vii. Easy Integration:

Can be integrated with different systems, servers, and network devices.

Chapter 4

FEASIBILITY STUDY

A feasibility study is an important step in the development of the Security Information and Event Management (SIEM) system. It acts as the initial phase where different aspects of the proposed system are analysed to determine whether the project is practical, achievable, and beneficial for a business organization. This phase combines technical understanding, resource planning, cost estimation, and time management to evaluate the overall viability of the project.

The main purpose of this feasibility study is to identify potential challenges and ensure that the SIEM system can be successfully implemented. It begins by assessing the technical requirements such as knowledge of cybersecurity concepts, log management, and SIEM tools like ELK Stack or Wazuh. It also identifies the roles of team members and evaluates both quantitative factors such as hardware, software, and cost, and qualitative factors such as system usability, reliability, and user acceptance.

As part of the system analysis, feasibility is further examined to ensure that the proposed SIEM solution aligns with organizational needs and available resources. The system requires basic hardware such as laptops or virtual machines and software tools like Elasticsearch, Logstash, Kibana, and log collection agents. These tools are open-source, making the system cost-effective and suitable for academic implementation. The availability of these resources confirms that the project is technically feasible.

A cost-benefit analysis is also considered in this study. Since the project uses open-source tools, the cost involved is minimal, while the benefits include improved security monitoring, real-time threat detection, and better incident response. This makes the system highly beneficial compared to its cost.

The feasibility study also evaluates different approaches to implementing the SIEM system and selects the most suitable method based on simplicity, efficiency, and practicality. A basic architecture using a centralized SIEM server and client systems is chosen for easy implementation and demonstration.

In addition to evaluating the basic viability of the proposed system, the feasibility study also focuses on ensuring that the Security Information and Event Management (SIEM) system can operate efficiently under real-world conditions. With the increasing complexity of cyber

threats, organizations require systems that are not only effective but also adaptable to evolving security challenges. The feasibility study helps in determining whether the system can meet these expectations while remaining cost-effective and manageable.

One of the key aspects considered during feasibility analysis is the technical capability of the system. The SIEM system relies on technologies such as log collection agents, centralized servers, and data visualization tools. These components must work together seamlessly to provide accurate and real-time monitoring of security events. Since the project utilizes open-source tools like ELK Stack (Elasticsearch, Logstash, Kibana) and Wazuh, it ensures that the system is built on reliable and widely supported technologies. These tools are well-documented and have strong community support, which makes implementation and troubleshooting easier.

Another important factor is the availability of resources. The SIEM system does not require highly specialized hardware, making it suitable for academic and small-scale organizational environments. Basic computing devices such as laptops or virtual machines are sufficient to deploy the system. This reduces the need for expensive infrastructure and ensures that the project can be implemented within limited resources. Additionally, the software tools used are freely available, eliminating licensing costs and making the system economically feasible.

The feasibility study also evaluates the skill requirements for implementing the system. The project requires basic knowledge of operating systems, networking, and cybersecurity concepts. Familiarity with tools such as Elasticsearch, Logstash, and Kibana is beneficial but not mandatory, as these tools are user-friendly and supported by extensive online documentation. This makes it possible for students and beginners to successfully implement the system with minimal training.

From an operational perspective, the SIEM system is designed to be user-friendly and easy to manage. The dashboard interface provides a clear and intuitive view of system activities, alerts, and logs. This allows users to quickly understand the security status of the organization without requiring advanced technical expertise. The system also supports real-time monitoring, enabling security analysts to respond to threats promptly. This improves overall operational efficiency and ensures that security incidents are handled effectively.

The feasibility study also considers the scalability of the system. As organizations grow, the number of devices and users increases, leading to a higher volume of security data. The SIEM system is designed to handle this growth by allowing additional log sources to be integrated.

Design And Implementation of Security Information and Event Management (SIEM) System for Monitoring Security Events in A Business Organization

The centralized architecture ensures that new systems can be added without significantly affecting performance. This makes the system suitable for both small and large organizations.

Another critical aspect of feasibility is the reliability and performance of the system. The SIEM system must be capable of processing large volumes of data without delays or failures. By using efficient data processing tools and indexing mechanisms, the system ensures fast data retrieval and analysis. This is essential for real-time threat detection, where delays can lead to serious security risks. The system is also designed to minimize false positives, ensuring that alerts are accurate and meaningful.

The economic feasibility of the SIEM system is one of its strongest advantages. Since the project is based on open-source tools, the overall cost of implementation is significantly reduced. There are no licensing fees, and the hardware requirements are minimal. The benefits of the system, such as improved security, reduced risk of cyber-attacks, and efficient incident response, far outweigh the initial setup costs. This makes the project a cost-effective solution for organizations looking to enhance their cybersecurity infrastructure.

In addition to cost savings, the system also helps in reducing potential financial losses caused by security breaches. Cyber-attacks can result in data loss, system downtime, and reputational damage, all of which can have significant financial implications. By detecting threats early and enabling quick response, the SIEM system helps organizations prevent such losses and maintain business continuity.

The feasibility study also includes a risk analysis, which identifies potential challenges that may arise during the implementation of the system. These challenges include handling large volumes of data, configuring detection rules, and managing false alerts. However, these issues can be addressed through proper system configuration, regular updates, and continuous monitoring. The use of well-established tools further reduces the risk of system failure.

Another important consideration is the legal and compliance feasibility. Many organizations are required to comply with data security regulations and standards. The SIEM system helps in maintaining detailed logs and generating reports that can be used for auditing and compliance purposes. This ensures that organizations meet regulatory requirements while maintaining a secure environment.

The feasibility study also evaluates the time required for implementation. Since the project uses pre-built tools and frameworks, the development time is relatively short. The system can be set

up and configured within a limited time frame, making it suitable for academic projects and quick deployment scenarios. Proper planning and task allocation further ensure timely completion of the project.

Furthermore, the SIEM system supports automation, which enhances its feasibility in modern environments. Automated log collection, analysis, and alert generation reduce the need for manual intervention. This not only improves efficiency but also ensures consistent performance. Automation also helps in handling large-scale data without increasing the workload on security teams.

The user acceptance of the system is another key factor in feasibility. The SIEM system is designed to be simple and easy to use, with a focus on providing clear and actionable insights. The dashboard interface allows users to interact with the system, making it suitable for both technical and non-technical users. This increases the likelihood of successful adoption within an organization.

In addition, the system provides flexibility in configuration, allowing organizations to customize rules and settings based on their specific requirements. This adaptability ensures that the system can be tailored to different environments and use cases. It also allows organizations to update the system as new threats emerge, ensuring long-term usability.

The feasibility study also highlights the long-term benefits of implementing a SIEM system. These include improved security awareness, better incident management, and enhanced decision-making capabilities. By providing a centralized view of security events, the system enables organizations to identify trends and patterns, helping them develop effective security strategies.

In conclusion, the feasibility study confirms that the proposed SIEM system is technically feasible, economically viable, and operationally efficient. The use of open-source tools, minimal hardware requirements, and user-friendly interfaces makes the system practical and easy to implement. The benefits of improved security monitoring, real-time threat detection, and efficient incident response make the project highly valuable for business organizations.

Overall, the SIEM system provides a reliable and scalable solution for managing security events in modern IT environments. The feasibility analysis ensures that the system can be successfully implemented within the available resources and constraints, making it a suitable and effective cybersecurity solution. Another important aspect considered in the feasibility

study is the performance efficiency of the SIEM system. Since the system deals with continuous log generation from multiple sources, it must be capable of handling high volumes of data without affecting performance. The use of efficient indexing and search mechanisms ensures that the system can quickly retrieve and analyse logs. This is especially important in real-time threat detection, where delays can reduce the effectiveness of security measures. The system is designed to maintain optimal performance even as the data volume increases.

The feasibility study also evaluates the security of the SIEM system itself. Since the system handles sensitive security data, it must be protected from unauthorized access. Proper authentication mechanisms, access control policies, and secure communication channels are required to ensure data confidentiality and integrity. By implementing these security measures, the system ensures that only authorized users can access and manage security data.

In addition, the study considers the maintainability of the system. A good system should be easy to maintain and update over time. The SIEM system is built using modular components, allowing individual modules to be updated or modified without affecting the entire system. This modular approach simplifies maintenance and ensures long-term usability. Regular updates and monitoring further help in keeping the system secure and efficient.

The usability of the system is another key factor in feasibility analysis. The SIEM system is designed with a user-friendly interface that allows users to easily navigate through dashboards, logs, and alerts. Even users with basic technical knowledge can understand the system and perform monitoring tasks effectively. This reduces the need for extensive training and increases system adoption within the organization.

Furthermore, the system supports real-time decision-making, which is essential in modern cybersecurity environments. By providing instant alerts and detailed information about security events, the system enables organizations to take immediate action. This helps in minimizing the impact of security incidents and ensures business continuity.

The feasibility study also takes into account the integration capability of the SIEM system. Modern organizations use multiple tools and technologies, and the SIEM system must be able to integrate with these systems. The use of standard protocols and APIs allows seamless integration with various devices, applications, and security tools. This ensures that the system can collect and analyse data from diverse sources.

Another critical factor is the data management capability of the system. The SIEM system stores large amounts of log data, which must be organized and managed efficiently. The centralized storage approach ensures that all data is available in one place, making it easier to analyse and retrieve information. This also supports historical analysis, which is useful for identifying long-term trends and improving security strategies.

The feasibility study also evaluates the reliability of the system under different conditions. The SIEM system must function properly even during peak loads or unexpected situations. By using reliable tools and proper configuration, the system ensures consistent performance. Backup mechanisms and data redundancy can also be implemented to prevent data loss and ensure system availability.

Another aspect considered is the flexibility of the system. The SIEM system can be customized based on organizational requirements. Security rules, alert thresholds, and monitoring parameters can be adjusted according to specific needs. This flexibility allows organizations to tailor the system to their environment and improve its effectiveness.

The feasibility study also highlights the importance of continuous improvement. Cyber threats are constantly evolving, and the SIEM system must be updated regularly to handle new types of attacks. By updating detection rules and monitoring strategies, the system can remain effective over time. This ensures long-term sustainability and relevance of the project.

Additionally, the system provides support for incident documentation and reporting. All detected events and alerts can be recorded and used to generate reports. These reports help in analyzing system performance, understanding security trends, and improving future strategies. This feature is particularly useful for auditing and compliance purposes.

The feasibility study also considers the impact of the system on organizational workflow. The SIEM system is designed to integrate smoothly into existing processes without causing disruptions. It enhances existing security measures rather than replacing them completely. This ensures that organizations can adopt the system without major changes to their infrastructure.

Another important consideration is the learning curve associated with the system. Since the project uses widely known tools and technologies, users can quickly learn and adapt to the system. The availability of online tutorials, documentation, and community support further simplifies the learning process.

The study also evaluates the long-term sustainability of the system. Open-source tools ensure that the system remains cost-effective and accessible. Organizations can continue using and upgrading the system without incurring additional costs. This makes the project sustainable in the long run.

Moreover, the SIEM system supports proactive security management. Instead of reacting to security incidents after they occur, the system helps in identifying potential threats in advance. This proactive approach reduces risks and improves overall security.

The feasibility study also examines the impact on organizational productivity. By automating security monitoring and analysis, the system reduces manual workload and allows employees to focus on other important tasks. This improves overall efficiency and productivity.

Finally, the feasibility study concludes that the SIEM system is not only feasible but also highly beneficial for modern organizations. It provides a comprehensive solution for monitoring, detecting, and responding to security threats. The combination of low cost, high efficiency, scalability, and reliability makes it an ideal choice for implementing cybersecurity solutions.

4.1 TECHNICAL FEASIBILITY

Technical feasibility focuses on evaluating whether the available technological resources, tools, and technical expertise are sufficient to successfully design and implement the Security Information and Event Management (SIEM) system within the given time frame and budget. It plays a key role in determining whether the system requirements, such as log collection, real-time monitoring, event correlation, and alert generation, can be supported by the existing infrastructure and tools.

For this project, technical feasibility begins with analysing the hardware and software requirements needed to implement the SIEM system. The system requires basic computing resources such as laptops or virtual machines to run the SIEM server and client systems. It also requires software tools such as ELK Stack (Elasticsearch, Logstash, Kibana) or Wazuh for log management, analysis, and visualization. These tools are capable of handling tasks such as log collection, processing, storage, and real-time threat detection efficiently.

Additionally, the technical feasibility study considers the skills and knowledge of the development team. The successful implementation of the project requires understanding of basic cybersecurity concepts, networking, and tools used in SIEM systems. Knowledge of

operating systems like Linux and Windows, along with basic configuration skills, is sufficient to develop this system. If there are any gaps in technical knowledge, they can be addressed through online resources, documentation, and practice.

Another important factor is the reliability and stability of the technologies used. The tools used in this project are open-source, widely used, and well-documented. They have strong community support, which makes troubleshooting and development easier. This ensures that any technical issues encountered during development or implementation can be resolved efficiently.

Overall, the technical feasibility study confirms that the proposed SIEM system is technically achievable using the available tools, resources, and team expertise. The project can be successfully implemented within the given constraints and provides a reliable solution for monitoring and managing security events in a business organization.

4.2 OPERATIONAL FEASIBILITY

Operational feasibility evaluates whether the Security Information and Event Management (SIEM) system will function effectively in a real-world business environment, meeting the security needs of the organization and improving overall system monitoring. It focuses on how well the system aligns with user expectations, how easily it can be adopted, and how efficiently it can operate after deployment.

Identifying Key User Needs:

The system addresses a critical requirement of organizations, which is continuous monitoring of security events and early detection of cyber threats. This helps in preventing security breaches and ensures timely response to incidents.

Solution Acceptance:

The proposed SIEM solution, built using open-source tools such as ELK Stack or Wazuh, is practical and widely accepted in cybersecurity environments. It provides real-time monitoring, alert generation, and easy visualization through dashboards, making it suitable for security teams.

User Adaptability:

The system is designed with user-friendly dashboards (Kibana/Wazuh), allowing users with basic technical knowledge to easily monitor logs and alerts. Minimal training is required for users to understand and operate the system effectively.

Organizational Fit:

The system aligns well with the goals of business organizations aiming to improve cybersecurity, monitor system activities, and protect sensitive data. It supports Security Operations Centre (SOC) activities and enhances overall security management within the organization.

4.3 ECONOMIC FEASIBILITY

Economic feasibility evaluates whether the Security Information and Event Management (SIEM) system is a financially viable project that can provide long-term benefits to a business organization. It involves analysing the cost of development, resource utilization, and the overall value the system delivers in terms of improved security, reduced risks, and efficient incident management.

Development Cost vs Long-Term Benefits:

Although there are initial costs involved in setting up the SIEM system, such as system configuration, tool installation, and basic infrastructure setup, the system significantly reduces the risk of security breaches and minimizes potential financial losses. It also reduces manual monitoring efforts, saving time and operational costs in the long run.

Resource Investment:

The project requires basic hardware such as laptops or virtual machines and software tools like ELK Stack (Elasticsearch, Logstash, Kibana) or Wazuh. Since these tools are open-source, the cost of implementation is very low while still providing high efficiency and performance.

Cost-Effective Solution:

By using open-source technologies and minimal hardware requirements, the overall cost of the system remains affordable. At the same time, it delivers high value by improving security monitoring, threat detection, and incident response capabilities.

Return on Investment (ROI):

Design And Implementation of Security Information and Event Management (SIEM) System for Monitoring Security Events in A Business Organization

The SIEM system enhances organizational security by detecting threats early and preventing major cyber incidents. This leads to cost savings by avoiding data breaches, system downtime, and financial losses, thereby providing a good return on investment.

Furthermore, the maintenance cost of the SIEM system is relatively low due to the use of open-source tools and minimal hardware requirements. The system does not require expensive licenses or high-end infrastructure, making it suitable for small and medium-sized organizations as well. Training costs are also minimal since the system uses user-friendly dashboards and widely available documentation. Over time, the system helps organizations avoid financial losses caused by cyber-attacks, data breaches, and system failures. Therefore, the investment made in implementing the SIEM system is justified by its ability to enhance security, reduce risks, and improve operational efficiency.

Chapter 5

SYSTEM ANALYSIS

5.1 EXISTING SYSTEM

The existing system for security monitoring in business organizations primarily relies on separate and independent security tools such as firewalls, antivirus software, and intrusion detection systems. Each of these tools works individually to detect specific types of threats, but they do not provide a centralized view of security events. As a result, security analysts must manually check multiple systems to identify potential threats, which is time-consuming and inefficient.

Moreover, the current systems lack real-time correlation of events. Even if multiple suspicious activities occur across different systems, they are not analysed together, making it difficult to detect complex attacks such as brute force or insider threats. This limitation increases the risk of delayed threat detection and response.

Another major drawback of the existing system is the absence of centralized log management. Logs are stored separately in different devices, making it difficult to analyse and investigate security incidents effectively. This can lead to missed threats and increased vulnerability within the organization.

Therefore, there is a need for an improved system that can collect, monitor, and analyse security events in a centralized manner. A SIEM-based solution helps overcome these limitations by providing real-time monitoring, event correlation, and efficient incident response.

In conclusion, while traditional security systems provide basic protection, they lack integration, real-time analysis, and centralized control. This highlights the necessity for a more advanced and unified approach like SIEM for effective security management.

the lack of centralized monitoring, traditional security systems also suffer from limited visibility across the network infrastructure. Since each tool operates independently, it becomes difficult for security analysts to gain a complete understanding of what is happening within the organization. For example, a firewall may detect unusual traffic, while an antivirus system may identify malware activity, but without integration, these events are not linked together. This

fragmented approach reduces the effectiveness of threat detection and increases the chances of missing critical security incidents.

Another major limitation of the existing system is the dependency on manual analysis. Security teams are required to review logs and alerts from multiple sources individually, which is both time-consuming and prone to human error. As the volume of data increases, it becomes nearly impossible for analysts to manually process all security events. This results in delayed responses and increases the risk of successful cyber-attacks. Manual monitoring also reduces efficiency and places a heavy burden on security personnel.

The existing systems also face challenges related to handling large volumes of data. Modern organizations generate massive amounts of log data from various devices and applications. Traditional tools are not designed to handle such high data volumes effectively. This leads to performance issues, data overload, and difficulty in identifying relevant information. As a result, important security events may go unnoticed.

Another critical issue is the lack of advanced threat detection capabilities. Traditional systems are primarily designed to detect known threats based on predefined signatures or rules. However, modern cyber-attacks are more sophisticated and often involve unknown or zero-day vulnerabilities. Existing systems are not capable of detecting such advanced threats, making organizations vulnerable to new and evolving attack techniques.

The absence of real-time monitoring and response is another drawback of the existing system. In many cases, security events are analysed after they occur, rather than in real time. This delay can have serious consequences, as attackers may exploit vulnerabilities before they are detected. Real-time monitoring is essential for identifying and responding to threats, but traditional systems fail to provide this capability.

Another limitation is the difficulty in incident investigation and analysis. Since logs are stored in different locations, it becomes challenging to trace the sequence of events during a security incident. Analysts must collect data from multiple sources and manually correlate them to understand the attack. This process is time-consuming and may lead to incomplete or inaccurate analysis.

The existing systems also lack automation capabilities, which are essential for efficient security management. Most traditional tools require manual configuration and intervention for detecting and responding to threats. This increases the workload on security teams and slows

down the response process. Automation can significantly improve efficiency, but it is not effectively implemented in traditional systems.

Another important drawback is the high maintenance and operational complexity. Managing multiple independent security tools requires significant effort and expertise. Each tool has its own configuration, updates, and maintenance requirements. This increases the overall complexity of the system and makes it difficult to manage efficiently.

The existing systems also face issues related to scalability. As organizations grow, the number of devices, users, and applications increases. Traditional security tools may not be able to handle this growth effectively. Scaling these systems often requires additional resources and costs, making it less practical for expanding organizations.

Another key issue is the lack of proper alert management. Traditional systems often generate a large number of alerts, many of which may be false positives. This makes it difficult for security analysts to identify genuine threats. Excessive alerts can lead to alert fatigue, where important alerts may be ignored or overlooked.

The absence of centralized reporting and visualization is also a major limitation. Existing systems do not provide a unified dashboard to monitor security events. This makes it difficult for decision-makers to understand the overall security status of the organization. Visualization tools are essential for identifying patterns and trends, but they are not effectively utilized in traditional systems.

Another limitation is the inability to correlate events across multiple systems. Cyber-attacks often involve multiple steps and occur across different systems. Without correlation, these events appear unrelated and may not be detected as part of a larger attack. This reduces the effectiveness of threat detection and increases security risks.

The existing systems also lack support for compliance and auditing requirements. Organizations are required to maintain logs and reports for regulatory purposes. Traditional systems do not provide efficient mechanisms for generating and managing such reports, making compliance difficult.

Another drawback is the limited adaptability to new technologies. With the adoption of cloud computing, remote work, and IoT devices, the security landscape has become more complex.

Traditional systems are not designed to handle these modern environments effectively, leading to security gaps.

The existing systems also face challenges related to data integration. Logs from different sources may have different formats, making it difficult to combine and analyse them. This lack of standardization reduces the efficiency of data analysis.

Furthermore, traditional systems do not provide proactive security measures. They primarily focus on detecting threats after they occur, rather than preventing them. Proactive monitoring and analysis are essential for modern cybersecurity, but existing systems fall short in this area.

Another important limitation is the lack of collaboration between security tools. Since each tool operates independently, there is no communication or coordination between them. This reduces the overall effectiveness of the security infrastructure.

In addition, existing systems often require high operational costs due to licensing, maintenance, and resource requirements. This makes them less suitable for small and medium-sized organizations.

The existing systems also lack intelligent decision-making capabilities. They do not use advanced techniques such as data analytics or machine learning to improve threat detection. This limits their ability to adapt to new threats.

Another issue is the difficulty in monitoring distributed environments. With the rise of cloud services and remote work, systems are no longer confined to a single location. Traditional tools are not equipped to monitor such distributed environments effectively.

The existing systems also face challenges related to data redundancy and duplication. Since logs are stored separately, there may be duplication of data, which increases storage requirements and reduces efficiency.

Finally, the lack of a unified security strategy is one of the biggest drawbacks of traditional systems. Without integration and coordination, it is difficult to implement a comprehensive security approach.

5.1.1 Disadvantages of Existing System

There are several disadvantages in the existing security monitoring systems used in business organizations, which rely on separate and independent security tools:

Time-consuming and inefficient:

Security analysts must manually check logs from multiple systems such as firewalls, servers, and antivirus tools. This process is slow and inefficient, especially in large organizations.

Lack of centralized monitoring:

Logs are stored in different systems without a unified platform, making it difficult to monitor all activities in one place.

No real-time analysis:

Existing systems do not provide real-time event correlation, leading to delays in detecting security threats.

Difficulty in detecting complex attacks:

Since events are not correlated, advanced attacks like brute force or insider threats may go unnoticed.

Increased complexity for users:

Managing multiple tools separately increases complexity and requires more effort from security teams.

Risk of missing threats:

Important security events may be overlooked due to scattered data and lack of integration.

Poor scalability:

As the organization grows, managing multiple security tools becomes difficult and inefficient.

Fragmented data management:

Security data is stored in different locations, making analysis and investigation more challenging.

5.2 PROPOSED SYSTEM

The proposed system aims to design and implement a comprehensive Security Information and Event Management (SIEM) system to effectively monitor and manage security events in a business organization. The system focuses on collecting, analyzing, and correlating security

logs from multiple sources such as client systems, servers, and network devices. It provides a centralized platform that enables real-time monitoring and efficient threat detection.

In the proposed system, security events from different systems are collected and processed in a unified environment. This eliminates the need for analysing logs from multiple independent tools, thereby reducing complexity and saving time. By correlating events from various sources, the system can detect suspicious activities such as multiple failed login attempts, unauthorized access, and abnormal system behaviour more accurately.

Furthermore, the system supports real-time alert generation, which helps security analysts respond quickly to potential threats. It enhances the overall security posture by providing continuous monitoring and early detection of cyber-attacks. The use of dashboard visualization allows users to easily understand system activities, alerts, and trends, improving decision-making.

The proposed system also supports Security Operations Centre (SOC) activities by simulating Tier 1 and Tier 2 roles. Tier 1 analysts monitor alerts and perform initial analysis, while Tier 2 analysts conduct deeper investigation and validation of incidents.

The system is implemented using open-source tools such as ELK Stack (Elasticsearch, Logstash, Kibana) or Wazuh, along with log collection agents. These tools enable centralized log management, real-time analysis, and efficient alerting mechanisms.

By using this proposed approach, the SIEM system provides an integrated, scalable, and cost-effective solution for security monitoring. It improves efficiency, reduces response time, and helps organizations proactively manage security threats. In addition to the basic functionalities described above, the proposed SIEM system is designed to provide a comprehensive and integrated approach to cybersecurity management. Unlike traditional systems that operate in isolation, this system brings together multiple components into a single unified platform. This integration allows for seamless communication between different modules, improving overall efficiency and effectiveness.

One of the major strengths of the proposed system is its ability to perform real-time log aggregation and analysis. Logs generated from various devices such as servers, endpoints, firewalls, and network systems are continuously collected and processed. This ensures that security events are monitored instantly, enabling organizations to detect and respond to threats without delay.

The system also emphasizes event correlation, which is a critical feature in modern cybersecurity. Instead of analyzing logs individually, the SIEM system correlates multiple events across different systems to identify complex attack patterns. For example, a series of failed login attempts followed by a successful login from an unusual location may indicate a brute-force attack. Such patterns can be effectively detected through correlation, improving the accuracy of threat detection.

Another important feature of the proposed system is automated alert generation. When suspicious activities are detected, the system generates alerts and notifies security analysts immediately. These alerts are categorized based on severity levels, allowing analysts to prioritize critical threats. This reduces response time and ensures that important incidents are addressed promptly.

The system also supports dashboard visualization, which provides a graphical representation of security data. Through charts, graphs, and reports, users can easily understand system behaviour, identify trends, and monitor security events. This enhances decision-making and helps organizations take proactive measures to improve security.

In addition, the proposed system includes centralized log management, where all logs are stored in a single repository. This simplifies data analysis and makes it easier to investigate security incidents. Analysts can quickly search and retrieve historical data, which is useful for identifying recurring threats and improving security strategies.

The system is also designed with scalability in mind. As organizations grow, the number of devices and data sources increases. The SIEM system can easily accommodate this growth by integrating additional log sources without affecting performance. This makes it suitable for both small and large organizations.

Another key aspect is the flexibility of the system. The SIEM platform allows customization of rules, thresholds, and configurations based on organizational requirements. This ensures that the system can adapt to different environments and security needs.

The proposed system also enhances incident response capabilities. By providing detailed information about security events, including timestamps, source details, and event types, the system enables analysts to investigate incidents thoroughly. This helps in identifying the root cause of attacks and taking appropriate corrective actions.

Furthermore, the system supports continuous monitoring, ensuring that security events are tracked 24/7. This is essential in today's digital environment, where cyber threats can occur at any time. Continuous monitoring helps in early detection and prevention of attacks.

The use of open-source tools such as ELK Stack and Wazuh makes the system cost-effective and accessible. These tools are widely used and supported by large communities, making it easier to implement and maintain the system.

The proposed system also improves data accuracy and consistency. By standardizing log formats and ensuring proper data processing, the system provides reliable information for analysis. This is crucial for making informed security decisions.

Another important feature is automation, which reduces manual effort and increases efficiency. Tasks such as log collection, processing, and alert generation are automated, allowing security teams to focus on critical activities.

The system also supports compliance and auditing requirements by maintaining detailed logs and generating reports. This helps organizations meet regulatory standards and ensures accountability.

In addition, the proposed system provides enhanced visibility into organizational activities. By monitoring all systems from a centralized platform, organizations can gain a clear understanding of their security posture.

The system is also designed to be user-friendly, with an intuitive interface that allows users to easily navigate and interact with the system. This improves usability and reduces the need for extensive training.

5.2.1 Advantages of Proposed System

The proposed Security Information and Event Management (SIEM) system offers several advantages for improving security monitoring and incident management in a business organization. Some of the key advantages are as follows:

Centralized Security Monitoring:

The system collects logs from multiple sources into a single platform, allowing security analysts to monitor all activities in one place without switching between different tools.

Real-Time Threat Detection:

The SIEM system provides real-time monitoring and analysis of security events, enabling early detection of suspicious activities and potential cyber threats.

Improved Accuracy in Threat Detection:

By correlating events from different systems, the SIEM system can identify complex attack patterns that may be missed by individual tools.

Faster Incident Response:

The system generates alerts immediately when a threat is detected, allowing security teams to respond quickly and reduce potential damage.

Enhanced Visibility:

Dashboard visualization provides clear insights into system activities, alerts, and trends, helping organizations understand their security posture.

Scalability and Flexibility:

The system can be easily expanded by adding more log sources and rules, making it suitable for growing organizations.

Cost-Effective Solution:

The use of open-source tools reduces implementation and maintenance costs while still providing powerful security features.

Reduced Risk of Security Breaches:

By continuously monitoring and analysing events, the system helps prevent cyber-attacks and minimizes the risk of data loss and system compromise.

Centralized monitoring, real-time detection, fast alerts, improved security visibility, scalable architecture, cost-effective solution, efficient incident response, enhanced threat analysis.

Chapter 6

SYSTEM DESIGN

The system design of the Security Information and Event Management (SIEM) system plays a crucial role in building an efficient and reliable platform for monitoring and analysing security events within a business organization. The design phase includes log collection, data processing, storage, event correlation, and visualization to ensure effective threat detection and

Possibilities

Log Collection & Data Integration:

The system collects log data from multiple sources such as client systems, servers, and network devices. These logs include information related to user activities, login attempts, system processes, and network traffic. Log collection agents such as Wingbeat, File beat, or Wazuh Agent are used to gather data and send it to the centralized SIEM server in a structured format.

Data Processing & Normalization:

The collected logs may contain inconsistent or unstructured data. Therefore, they are processed and normalized using tools like Logstash or Wazuh decoders. This step involves parsing logs, extracting important fields (such as timestamp, IP address, username), and converting them into a standardized format for easy analysis.

Data Storage & Indexing:

After processing, the logs are stored in a centralized database such as Elasticsearch. The data is indexed efficiently to allow fast searching, filtering, and retrieval. This helps in analysing large volumes of security data in real time.

Event Correlation & Detection:

The SIEM system applies predefined rules to correlate events from different sources. It detects suspicious patterns such as multiple failed login attempts, unauthorized access, and abnormal system behaviour. This helps in identifying potential security threats more accurately.

Alert Generation & Monitoring: When suspicious activities are detected, the system generates alerts to notify security analysts. Real-time monitoring is performed through dashboards, enabling quick identification and response to incidents.

Visualization & Reporting:

The system provides dashboard visualization using tools like Kibana or Wazuh Dashboard. It displays logs, alerts, and system activity in graphical formats such as charts and graphs, improving understanding and decision-making

6.1 SYSTEM ARCHITECTURE

The below figure illustrates the architectural design of the Security Information and Event Management (SIEM) system. This architecture provides a summarized, top-level view of the project, showing how security events are collected, processed, analyzed, and monitored within a business organization. It highlights the key components and workflow involved in detecting and managing security threats in real time.

The system architecture of the SIEM system presents a complete workflow starting from log collection to threat detection and response. The architecture begins with collecting log data from various sources such as client systems, servers, and network devices. These logs are then forwarded to a centralized SIEM server where they are processed, analyzed, and visualized for monitoring purposes.

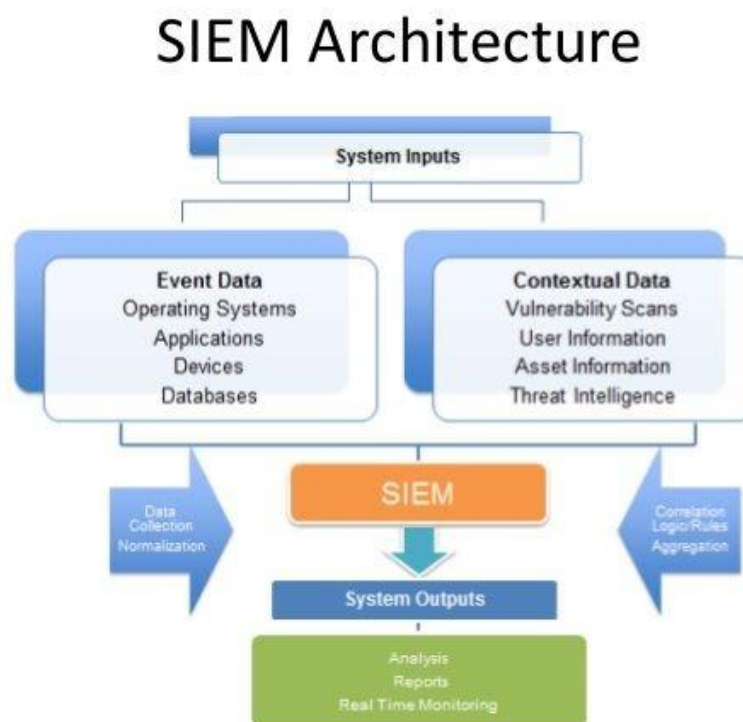


Fig. 6.1: System Architecture

1. Log Collection:

The first step in the system is to collect logs from multiple sources such as endpoints, servers, firewalls, and network devices. These logs include user activities, login attempts, system processes, and network traffic. Log collection agents such as Winlogbeat, Filebeat, or Wazuh Agent are used to gather and forward this data to the SIEM server.

2. Log Processing:

The collected logs are often unstructured and require processing. Tools like Logstash or Wazuh decoders are used to parse and structure the logs. Important information such as timestamps, IP addresses, usernames, and event types is extracted for further analysis.

3. Data Storage:

After processing, the logs are stored in a centralized database such as Elasticsearch. The data is indexed efficiently, allowing fast search and retrieval of large volumes of security data.

4. Event Correlation:

The system applies predefined rules to correlate events from different sources. It identifies patterns such as repeated failed login attempts, unauthorized access, and unusual system behaviour, which may indicate potential security threats.

5. Alert Generation:

When suspicious activities are detected, the system generates alerts. These alerts notify security analysts about potential threats, enabling quick action to prevent security breaches.

6. Dashboard Visualization:

The system provides visualization through dashboards using tools like Kibana or Wazuh Dashboard. It displays logs, alerts, and system activities in graphical formats such as charts and graphs, making it easier to analyse data.

7. Monitoring & Analysis (SOC):

Security analysts monitor the system using dashboards.

Tier 1 analysts handle alert monitoring and initial investigation

Tier 2 analysts perform deeper analysis and incident validation

6.2 UML DIAGRAMS

UML (Unified Modelling Language) diagrams serve as a standardized method to visualize and document the design of the Security Information and Event Management (SIEM) system. These diagrams are widely used by software engineers and system architects to represent system architecture, workflows, data flow, and interactions between different components. UML diagrams simplify complex system operations into clear visual representations, making it easier to understand, design, and maintain the system.

Importance of UML Diagrams in SIEM Project

In a SIEM system, multiple components work together to monitor and manage security events. The system involves continuous log collection, processing, threat detection, alert generation, and monitoring. These operations include several modules such as:

- Log collection from client systems
- Log processing and normalization
- Data storage and indexing
- Event correlation and detection
- Alert generation system
- Dashboard visualization and monitoring
- SOC analysis and response

UML diagrams help represent these components visually and show how they interact with each other. This makes it easier for developers, analysts, and stakeholders to understand the overall workflow of the SIEM system without needing deep technical knowledge.

Types of UML Diagrams Used in This Project

In this project, the following six UML diagrams are used to represent different aspects of the SIEM system:

Use Case Diagram – Shows interactions between users (SOC analysts) and the system

Class Diagram – Represents system structure, classes, and relationships

Sequence Diagram – Shows step-by-step interaction between components

Activity Diagram – Represents workflow of security monitoring process

Component Diagram – Shows system components like ELK, agents, and their connections

· **Deployment Diagram** – Represents physical deployment (server, client systems, network)

6.2.1 Use Case Diagram

A Use Case Diagram in the Unified Modelling Language (UML) is a type of behavioural diagram that represents the functional requirements of the Security Information and Event Management (SIEM) system. It provides a graphical overview of how different users (actors) interact with the system and the services (use cases) offered by the system. The main purpose of the use case diagram is to show what system functions are performed for each actor.

In the SIEM system, the primary actors include Security Analysts (Tier 1 and Tier 2), System Administrators, and Client Systems. Each actor interacts with the system to perform specific tasks such as monitoring logs, analysing alerts, managing system configurations, and generating reports. The diagram helps in clearly identifying the roles of each actor and their interaction with the system.

Use case diagrams provide a visual representation of the system's functionality and help in understanding how different components of the SIEM system work together. They also assist developers and stakeholders in analysing system behaviour, identifying relationships between use cases, and ensuring that all functional requirements are covered.

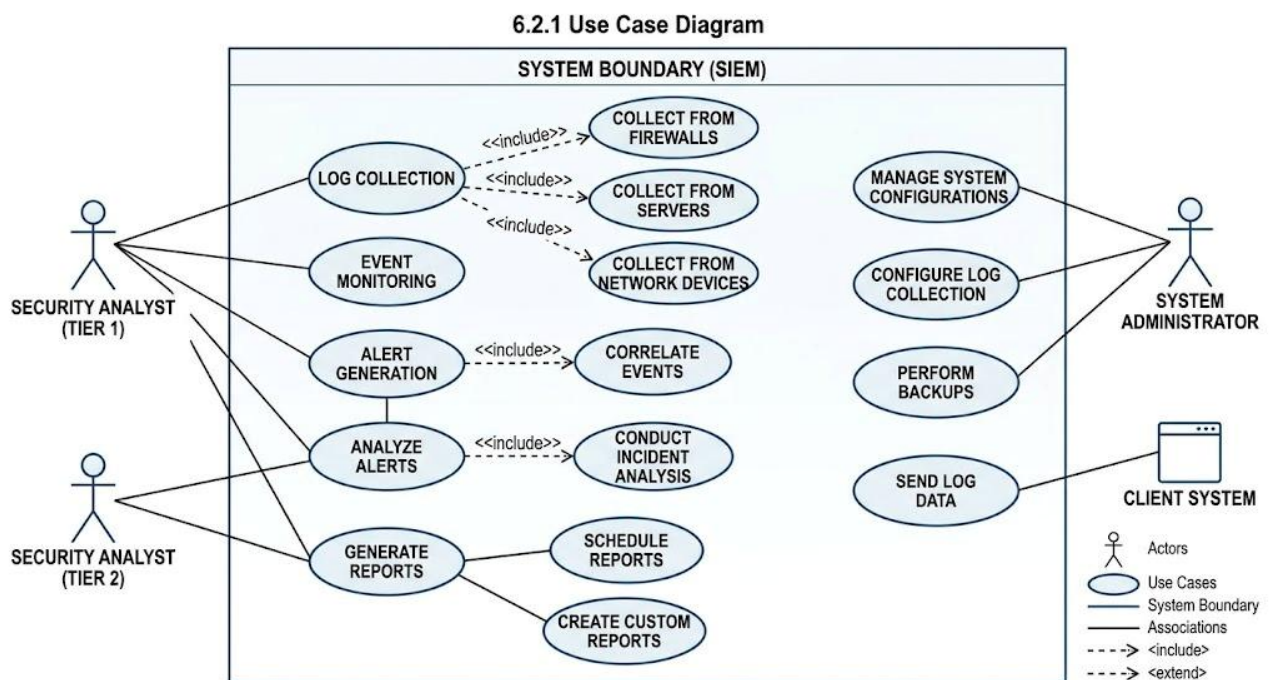


Fig. 6.2.1: Use Case Diagram

6.2.2 Class Diagram

A class diagram is a type of UML (Unified Modelling Language) diagram that represents the static structure of the Security Information and Event Management (SIEM) system by showing its classes, attributes, methods, and the relationships between them. It plays an important role in the design phase by providing a clear overview of how different components of the system are organized and interact with each other.

In the context of the SIEM system, the class diagram helps to represent key components such as Log, User, Alert, SIEM Server, Agent, Dashboard, and Analyst. Each class contains relevant attributes (such as timestamp, IP address, username, event type) and methods (such as collectLogs (), analyzeEvents (), generateAlert (), viewDashboard ()). These classes are connected based on their relationships, such as association, dependency, and inheritance.

The class diagram provides a structured representation of how data flows within the system and how different modules interact. For example, the Agent class collects logs and sends them to the SIEM Server, which processes and stores them. The Alert class is triggered when suspicious activity is detected, and the Analyst class interacts with the Dashboard to monitor and analyse events.

6.2.2 CLASS DIAGRAM: SIEM SYSTEM

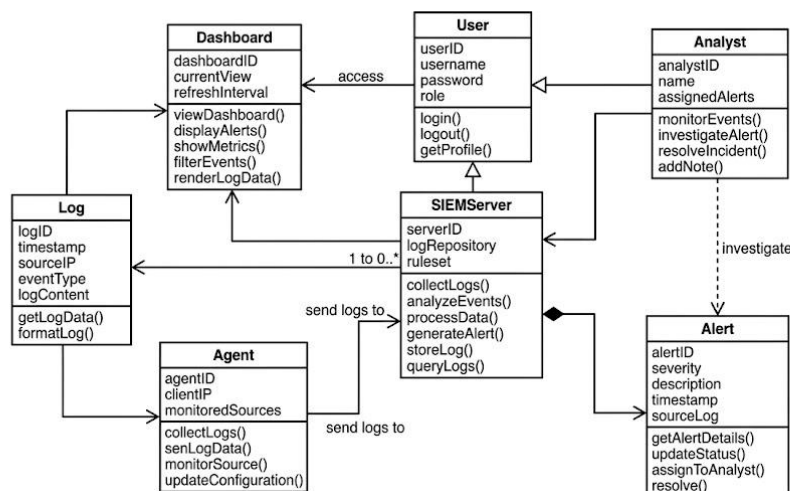


Fig. 6.2.2: Class Diagram

6.2.3 Sequence Diagram

A sequence diagram is a type of interaction diagram in Unified Modelling Language (UML) that illustrates how and in what order different components of the Security Information and Event Management (SIEM) system interact with each other. It shows the sequence of messages exchanged between objects to perform a specific function. These diagrams are useful for understanding system workflows and documenting how processes are executed step by step.

In the context of the SIEM system, the sequence diagram represents how data flows from client systems to the SIEM server and how it is processed and analysed. The interaction begins when the log collection agent gathers data from client systems and sends it to the SIEM server. The server processes the logs, stores them in the database, and applies correlation rules to detect suspicious activities.

If a threat is detected, the system generates an alert and sends it to the dashboard. The security analyst then views the alert and performs analysis. This sequence of interactions helps in understanding how different components such as agents, server, database, alert engine, and dashboard work together in a step-by-step manner.

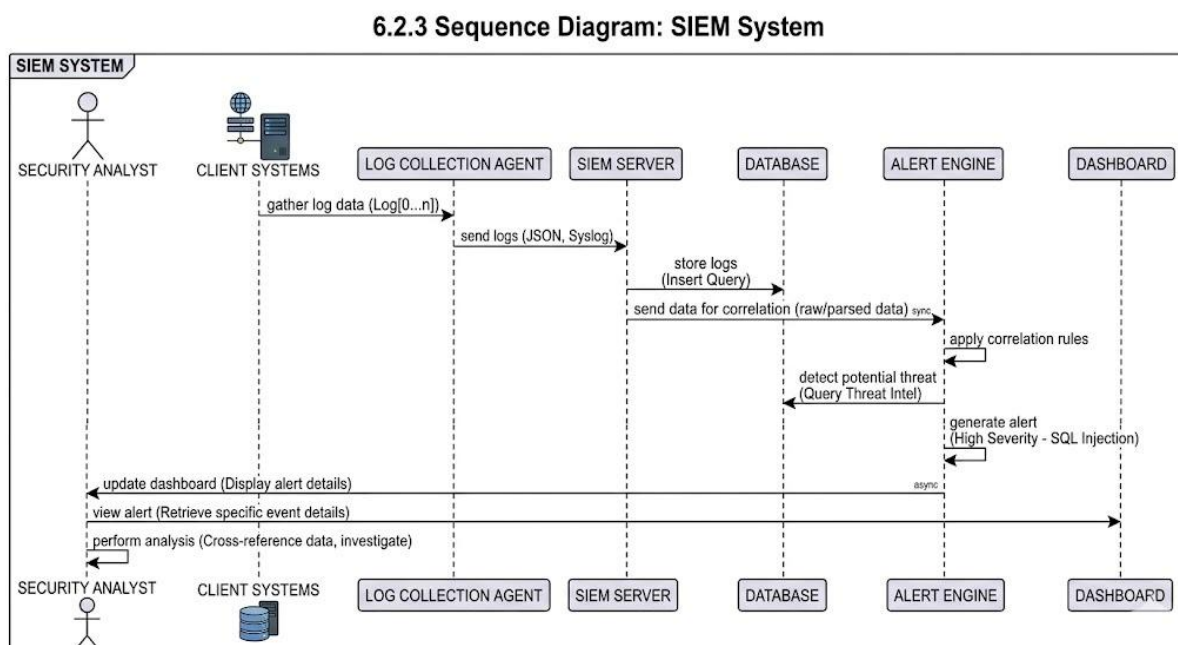


Fig. 6.2.3: Sequence Diagram

6.2.4 Activity Diagram

An activity diagram is a type of behavioural UML diagram that represents the dynamic workflow of the Security Information and Event Management (SIEM) system. It is mainly used to illustrate the sequence of activities involved in monitoring, detecting, and responding to security events. The diagram shows how different operations are carried out step by step, including decision points, parallel processes, and flow of control within the system.

In the context of the SIEM system, the activity diagram describes the complete workflow starting from log generation in client systems to threat detection and response. The process begins with log collection from various sources, followed by log processing and normalization. The system then analyses the data using correlation rules to identify suspicious activities.

Based on the analysis, a decision is made whether the activity is normal or malicious. If a threat is detected, an alert is generated and sent to the dashboard for monitoring. The security analyst then reviews the alert and performs further investigation. If necessary, appropriate actions are taken to mitigate the threat.

Activity diagrams clearly represent the flow of operations, decision-making steps, and interactions between different components, making it easier to understand the working process of the SIEM system.

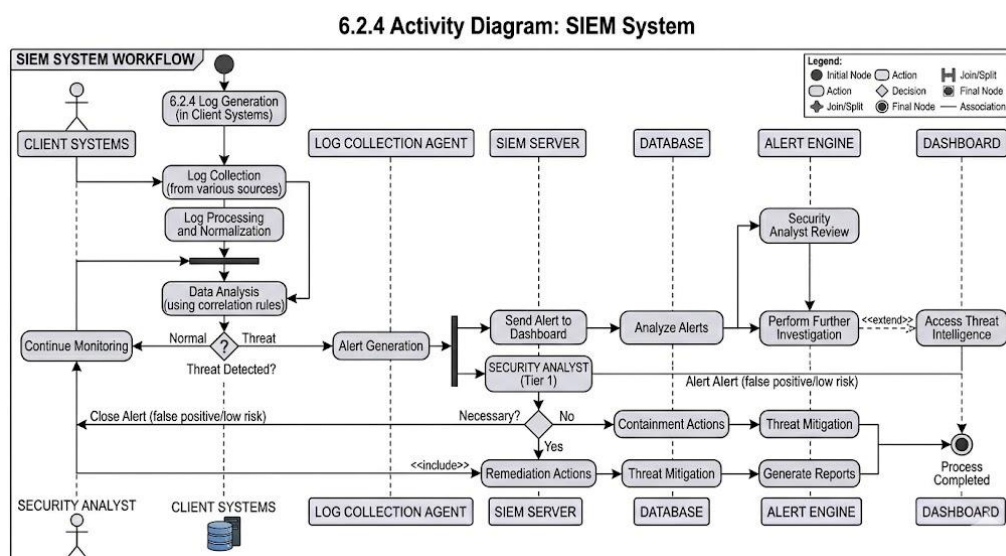


Fig. 6.2.4: Activity Diagram

6.2.5 Component Diagram

A component diagram is a type of UML structural diagram that is used to represent the physical components of the Security Information and Event Management (SIEM) system. Its main purpose is to divide the system into smaller, manageable modules, where each component represents a specific functionality of the system. These components can be developed independently and later integrated to form a complete working system. This helps developers understand the system structure and the interaction between different modules.

In the context of the SIEM system, the component diagram illustrates the major modules such as Log Collection Agents, Log Processing Unit, Data Storage (Elasticsearch), Event Correlation Engine, Alert Generation System, and Dashboard Interface (Kibana/Wazuh). Each component performs a specific function and communicates with other components through defined interfaces.

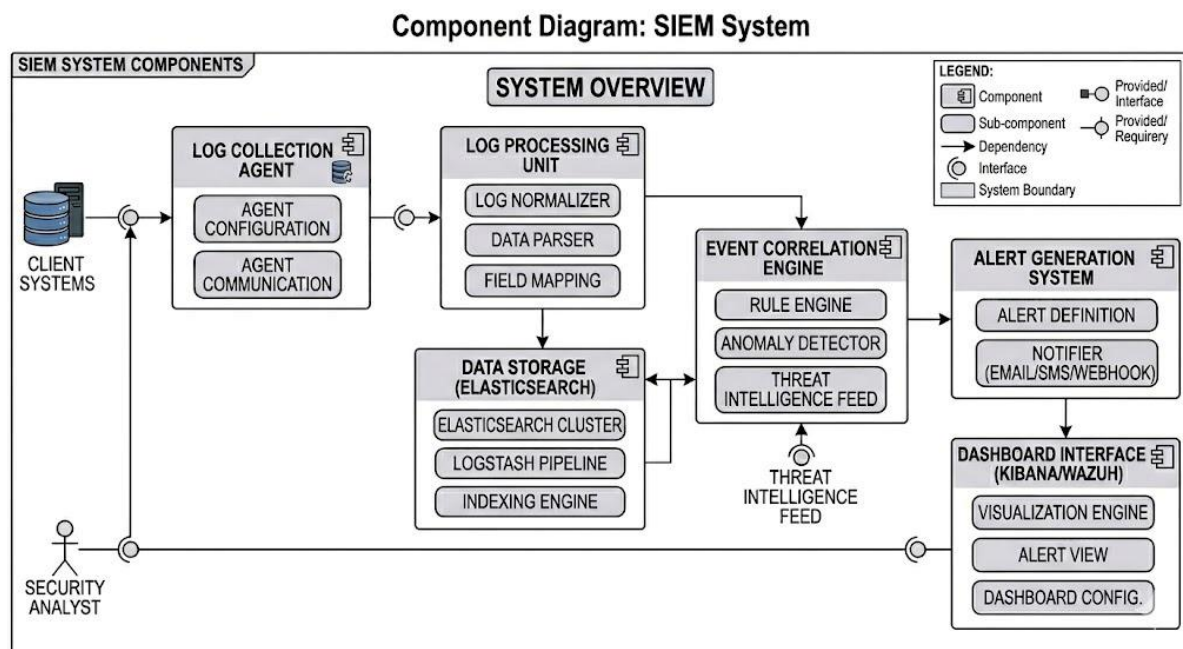


Fig. 6.2.5: Component Diagram

Chapter 7

SYSTEM IMPLEMENTATION

7.1 ALGORITHM

The process of security event monitoring in a SIEM system involves several steps. The first step is log collection, where data is gathered from various sources such as client systems, servers, and network devices. These logs contain information about user activities, login attempts, system events, and network traffic. The collected data is then pre-processed to ensure it is suitable for analysis.

Next, log processing and normalization are performed to convert raw data into a structured format. Important features such as timestamp, IP address, username, and event type are extracted. After preprocessing, event correlation is applied using predefined rules to identify suspicious patterns such as multiple failed login attempts or unauthorized access.

The system then analyzes these events and determines whether they are normal or malicious. If a threat is detected, an alert is generated and sent to the dashboard for monitoring. The performance of the system is evaluated based on its ability to detect threats accurately and respond in real time.

7.1.1 Event Correlation Algorithm

Event correlation is a key technique used in SIEM systems to analyse multiple events and detect security threats. It works by applying predefined rules to identify patterns across different logs.

Event correlation helps in:

- Detecting repeated failed login attempts
- Identifying unauthorized access
- Recognizing abnormal system behaviour

This method improves threat detection by combining multiple events instead of analysing them individually, making the system more efficient and accurate.

7.1.2 Rule-Based Detection

Rule-based detection is used to identify suspicious activities based on predefined conditions. The system checks incoming logs against a set of rules to determine whether an event is normal or malicious.

For example:

- If failed login attempts > 5 → Generate alert
- If unknown IP access detected → Flag as suspicious

This method provides quick and reliable detection of known threats.

7.1.3 Advanced Detection Techniques (Correlation & Aggregation)

In a SIEM system, advanced detection techniques are used to improve the accuracy of identifying security threats. Techniques such as correlation and aggregation help in combining multiple security events to detect complex attack patterns. These methods work by analysing logs collected from different sources and identifying relationships between them.

Correlation-based detection analyzes multiple events together instead of individually. For example, if multiple failed login attempts occur within a short period, the system identifies it as a brute force attack. Aggregation techniques group similar events to reduce noise and improve clarity in analysis.

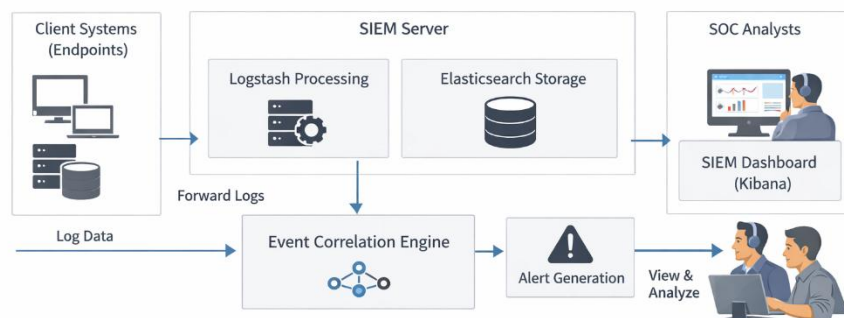


Fig. 7.1.3: Advanced Detection Techniques

7.1.4 Rule-Based Detection System

The rule-based detection system is a core component of the SIEM system that identifies suspicious activities using predefined rules. These rules are created based on known attack patterns and security policies.

For example, if the number of failed login attempts exceeds a certain threshold, the system generates an alert. Similarly, access from unknown IP addresses or unusual system behaviour is flagged as suspicious.

The rule-based system continuously monitors incoming logs and compares them with defined conditions. If any condition is satisfied, an alert is triggered.

Key Features:

Fast detection of known threats

- Easy to configure and update rules
- Supports real-time monitoring
- Improves incident response time

7.2 WORKING AND MODULE EXPLANATION

The SIEM system works by collecting, processing, analysing, and monitoring security events generated from different systems within an organization. The process is based on continuous log monitoring and analysis to detect suspicious activities and respond to potential threats.

The first step is log collection, where data is gathered from multiple sources such as endpoints, servers, and network devices. These logs contain important information such as user activity, login attempts, system processes, and network traffic.

The collected logs are then processed and normalized to convert them into a structured format. This includes extracting important fields such as timestamp, IP address, username, and event type. After processing, the logs are stored in a centralized database.

Next, the system applies event correlation and rule-based detection techniques to analyse the logs. It identifies patterns that indicate potential threats such as unauthorized access, multiple failed logins, or abnormal behaviour.

If a suspicious activity is detected, the system generates an alert and displays it on the dashboard. Security analysts monitor these alerts and perform further investigation. Based on the analysis, appropriate actions are taken to prevent or mitigate the threat.

Overall, the SIEM system provides continuous monitoring, real-time detection, and efficient incident response, improving the overall security of the organization.

7.2.1 Modules Involved

This project mainly consists of 6 modules. They are:

1. Log Collection and Processing
2. Log Analysis and Correlation
3. Alert Generation System
4. Dashboard and Monitoring
5. Data Storage and Management
6. SOC Analysis and Response

The SIEM system follows a structured process beginning with log collection, where data is gathered from client systems and network devices using agents. Since raw logs may be unstructured, they go through processing and normalization where irrelevant or duplicate data is removed and the logs are formatted properly.

This is followed by log analysis, where important features such as user activity, IP address, and event type are extracted. Event correlation techniques are applied to identify patterns and detect suspicious activities.

Next comes alert generation, where the system creates alerts based on predefined rules. These alerts are displayed on dashboards, allowing analysts to monitor system activity in real time.

The system also includes a data storage module, where logs are stored securely for future analysis and investigation. Finally, the SOC analysis module allows security analysts to investigate alerts, validate incidents, and take appropriate action.

CHAPTER 8

UNIT TESTING

8.1 UNIT TESTING

Unit testing in the context of the Security Information and Event Management (SIEM) system refers to testing the smallest functional components of the system individually to ensure they work correctly. These components include log collection modules, log processing functions, event detection rules, and alert generation mechanisms. Unit testing is typically performed during the development phase to verify that each module behaves as expected.

For example, unit testing is performed on the log collection module to ensure that logs are correctly captured from client systems and transmitted to the SIEM server. Similarly, the log processing function is tested to verify that it properly parses and normalizes raw log data. The event correlation module is tested to ensure that it accurately detects suspicious patterns such as multiple failed login attempts.

Additionally, the alert generation module is tested to confirm that alerts are triggered correctly based on predefined rules. Each function is tested independently to identify and fix errors early in the development process. This helps improve system reliability and ensures that all components function properly before integration.

This process helps in identifying bugs at an early stage, ensuring that each internal function—such as log collection, data processing, event correlation, and alert generation—is producing accurate results for given inputs. Since the SIEM system involves decision-making based on rule-based detection and event correlation, unit testing becomes essential to validate different detection scenarios and logic flows.

For example, testing is performed to verify whether the system correctly detects multiple failed logins attempts or identifies unauthorized access patterns. Before integrating modules like log processing, alert generation, and dashboard visualization into the complete system, each module is tested independently to ensure reliability and correctness.

This step-by-step testing approach improves the overall system stability, reduces errors during integration, and ensures that the SIEM system performs efficiently in real-time security monitoring and threat detection. In addition to verifying individual components, unit testing also plays a crucial role in ensuring the overall quality and robustness of the SIEM system.

Since the system is responsible for handling critical security data, even a small error in one module can lead to incorrect analysis or missed detection of threats. Therefore, thorough unit testing is necessary to validate each function and ensure that it performs accurately under different conditions.

One of the important areas of unit testing in the SIEM system is log data validation. Logs collected from various sources may contain incomplete, inconsistent, or unexpected values. Unit testing ensures that the system can handle such variations without failure. For example, tests are conducted to verify how the system behaves when log entries are missing fields such as timestamps or IP addresses. This ensures that the system remains stable even when dealing with imperfect data.

Another critical aspect is testing log parsing and normalization functions. Since logs from different systems come in various formats, the SIEM system must convert them into a standardized structure. Unit testing verifies whether the parsing logic correctly extracts key fields such as event type, source IP, destination IP, and user information. It also checks whether the normalization process maintains data consistency across different log sources.

Unit testing is also essential for validating event correlation logic. The SIEM system relies on predefined rules to detect suspicious patterns by analyzing multiple events. These rules must be tested thoroughly to ensure that they correctly identify security threats. For instance, unit tests are conducted to simulate scenarios such as repeated failed login attempts, unusual access times, or access from unknown locations. These tests help confirm that the system accurately detects such patterns and triggers appropriate alerts.

In addition, alert generation mechanisms are tested to ensure their correctness and reliability. Unit testing verifies whether alerts are generated only when required conditions are met and ensures that false alerts are minimized. This is important because excessive false alerts can overwhelm security analysts and reduce the effectiveness of the system. By testing alert thresholds and conditions, the system can be fine-tuned to provide accurate and meaningful alerts.

Another area covered in unit testing is error handling and exception management. The SIEM system must be capable of handling unexpected errors without crashing. Unit tests are designed to simulate error conditions such as invalid input data, network failures, or system

interruptions. These tests ensure that the system can recover gracefully and continue functioning without affecting overall performance.

The performance of individual modules is also evaluated during unit testing. Each component is tested to ensure that it can handle the expected workload efficiently. For example, the log processing module is tested to verify that it can process a large number of log entries within a short time. This ensures that the system can support real-time monitoring without delays.

Unit testing also focuses on data accuracy and consistency. Since the SIEM system is used for security analysis, it is important that the data processed and stored by the system is accurate. Unit tests verify that there is no data loss or corruption during processing. They also ensure that the data displayed on dashboards matches the actual log data.

Another important aspect of unit testing is testing individual modules in isolation. Each module is tested independently without considering its interaction with other modules. This helps in identifying issues at the earliest stage and simplifies debugging. Once all modules are tested individually, they can be integrated with confidence that they will work correctly together.

The SIEM system also includes dashboard and visualization components, which are tested to ensure that they correctly display data. Unit testing verifies whether charts, graphs, and reports accurately represent the underlying data. It also checks whether the user interface responds correctly to user actions such as filtering logs or selecting time ranges.

In addition, unit testing helps in validating the configuration settings of the system. The SIEM system allows customization of rules, thresholds, and parameters. Unit tests ensure that these configurations are applied correctly and that changes in settings do not affect system functionality negatively.

Another important benefit of unit testing is that it supports continuous development and improvement. As new features are added to the system, unit tests can be used to verify their correctness. This ensures that new changes do not introduce errors into existing functionality. It also helps maintain the stability of the system over time.

Unit testing also plays a significant role in reducing development time and cost. By identifying and fixing errors early in the development process, it prevents issues from becoming more complex and costly to resolve later. This improves overall development efficiency and ensures timely completion of the project.

Furthermore, unit testing improves the confidence of developers and users in the system. When each component is thoroughly tested, it ensures that the system performs reliably under different conditions. This builds trust in the system and increases its acceptance among users.

The use of automated testing tools can further enhance the effectiveness of unit testing. Automation allows repetitive tests to be executed quickly and consistently, reducing manual effort and improving accuracy. Automated tests can also be integrated into the development process, enabling continuous testing as the system evolves.

Unit testing also supports documentation and understanding of the system. Each test case provides information about how a particular function is expected to behave. This helps developers understand the system better and makes it easier to maintain and update the system in the future.

In the context of the SIEM project, unit testing is particularly important because of the critical nature of security monitoring. Any failure in detecting threats can have serious consequences. Therefore, thorough testing of each module is essential to ensure that the system performs its intended function effectively.

The testing process also includes boundary value testing, where the system is tested with extreme or unusual inputs. For example, testing may involve extremely high numbers of login attempts or unusual log patterns. This ensures that the system can handle edge cases without failure.

Another important testing approach is positive and negative testing. Positive testing verifies that the system works correctly with valid inputs, while negative testing ensures that the system handles invalid inputs appropriately. Both types of testing are essential for ensuring system reliability.

Unit testing also ensures that the system maintains consistency across different environments. The system may be deployed on different machines or configurations, and unit tests help verify that it behaves consistently in all scenarios.

Additionally, unit testing helps in identifying performance bottlenecks in the system. By analyzing the performance of individual modules, developers can optimize the system for better efficiency. This is particularly important in SIEM systems, where real-time performance is critical.

The SIEM system also benefits from regression testing, which ensures that previously tested functionalities continue to work correctly after changes are made. This prevents new updates from introducing unexpected issues.

In conclusion, unit testing is a fundamental step in the development of the SIEM system. It ensures that each component functions correctly, improves system reliability, and reduces the risk of errors. By thoroughly testing individual modules, the system can achieve high accuracy in detecting security threats and provide efficient real-time monitoring.

Overall, unit testing contributes significantly to the success of the SIEM project by ensuring that the system is robust, reliable, and capable of handling real-world security challenges effectively.

Another important dimension of unit testing in the SIEM system is the validation of real-time processing capabilities. Since the system continuously receives log data from multiple sources, it must be able to process incoming data without delay. Unit tests are designed to simulate continuous data flow and verify whether the system can process logs instantly without buffering issues or performance degradation. This ensures that the SIEM system remains responsive even under heavy workloads.

Unit testing also focuses on time-based event validation. Many security events depend on time intervals, such as detecting multiple failed login attempts within a specific duration. Tests are conducted to verify whether the system correctly handles time-based conditions and triggers alerts accurately. This ensures precise detection of attacks like brute force attempts and unusual access timings.

Another key area is testing data flow accuracy between functions. Each module in the SIEM system passes data to another module. Unit testing ensures that the output of one function becomes the correct input for the next function without any data loss or transformation errors. This guarantees smooth internal communication between modules.

The SIEM system also requires testing of log filtering mechanisms. Since not all logs are important, the system filters relevant data for analysis. Unit tests verify whether filtering rules correctly include necessary logs and exclude irrelevant ones. This improves system efficiency and reduces unnecessary processing.

Unit testing also ensures proper functioning of threshold-based conditions. For example, if a rule is set to trigger an alert after five failed login attempts, tests verify whether the alert is triggered exactly at the defined threshold. This prevents both under-detection and over-detection of threats.

Another crucial aspect is testing system resilience. The SIEM system must continue functioning even when unexpected situations occur. Unit tests simulate conditions such as corrupted log entries, missing data, or unexpected input formats to ensure the system can handle such cases without failure.

Unit testing also evaluates the accuracy of alert messages. It verifies that alerts contain correct and meaningful information such as event type, timestamp, and source details. This is important because security analysts rely on these alerts to make decisions.

In addition, testing is performed to ensure consistency in repeated operations. When the same input is given multiple times, the system should produce consistent outputs. Unit testing ensures that there are no variations in behaviour due to internal inconsistencies.

The SIEM system also requires testing of data storage accuracy. Unit tests verify that processed logs are correctly stored in the database without duplication or loss. This ensures reliable data availability for future analysis.

Another area of testing is response time validation. The system must generate alerts within an acceptable time frame. Unit tests measure the time taken for processing and alert generation to ensure it meets performance requirements.

Unit testing also includes validation of configuration changes. When system settings such as rules or thresholds are modified, tests ensure that the changes are applied correctly without affecting other functionalities.

Furthermore, unit testing supports secure coding practices. It helps identify vulnerabilities in individual modules, ensuring that the system is not only functional but also secure.

The testing process also ensures that the system handles multiple input scenarios effectively. Whether the input data is minimal, moderate, or large, the system should perform consistently. Unit testing verifies this adaptability.

Another key aspect is log format compatibility testing. Since logs may come from different systems, the SIEM system must support multiple formats. Unit tests verify compatibility with various log formats.

Unit testing also helps in validating rule optimization. Over time, detection rules may need refinement. Testing ensures that optimized rules improve accuracy without introducing errors.

Additionally, the system undergoes stress testing at the unit level, where individual modules are tested under extreme conditions to evaluate their limits. This helps ensure system stability under heavy load.

Unit testing also ensures proper functioning of data synchronization mechanisms. If multiple processes access the same data, tests verify that synchronization is maintained without conflicts.

Another benefit is improved debugging efficiency. Since each module is tested separately, identifying the source of errors becomes easier, reducing development time.

Unit testing also contributes to system documentation, as each test case defines expected behaviour, making the system easier to understand and maintain.

Finally, unit testing ensures that the SIEM system is robust, reliable, and efficient, capable of handling real-world cybersecurity challenges. It forms the foundation for higher-level testing and guarantees that the system performs accurately in detecting and responding to threats.

In addition to these aspects, unit testing also plays a vital role in ensuring **consistency and reliability across repeated executions** of the SIEM system. When the same inputs are provided multiple times, the system must produce identical outputs without variation. Unit tests are designed to verify this consistency, ensuring that the system behaves predictably under all conditions. This is especially important in security systems, where even minor inconsistencies can lead to incorrect threat detection.

Furthermore, unit testing supports the validation of **modular independence**, ensuring that changes in one module do not negatively affect other components. This helps maintain system stability during updates and enhancements. It also enables developers to confidently modify or improve specific modules without impacting the overall system performance.

Another important benefit is the ability to support **continuous integration and development practices**. As new features are added, unit tests can be reused to verify system stability.

**Design And Implementation of Security Information and Event Management (SIEM)
System for Monitoring Security Events in A Business Organization**

Table. 8.1: Test Cases

Function	Test Description	Input	Expected Output	Pass Criteria
Log Collection	Collect logs from server	Server log data	Logs successfully stored	Logs successfully received and stored in database
Log Processing	Test log parsing and normalization	Raw log files	Logs converted into structured format	Logs correctly formatted and stored
Event Detection	Detect suspicious login attempts	Multiple failed login attempts	Security event detected	System correctly identifies suspicious activity
Alert Generation	Trigger alert when a security rule is activated	Security event detected	Alert notification generated	Alert displayed in dashboard and notification sent
Database Storage	Store processed logs in database	Processed log data	Logs stored in centralized database	Logs successfully saved and retrievable

8.2 INTEGRATION TESTING

Integration testing in the Security Information and Event Management (SIEM) system focuses on verifying the interaction between different modules such as log collection, log processing, data storage, event correlation, and alert generation. It ensures that all these modules work together seamlessly as a single system. Although each module is individually tested during unit testing, integration testing confirms that the combined system functions correctly and efficiently.

In this project, integration testing is essential because multiple components need to interact properly. For example, logs collected from client systems must be correctly transmitted to the SIEM server, processed accurately, stored in the database, and then analyzed for threat detection. The integration tests verify that data flows smoothly between these modules without any loss or errors.

Additionally, integration testing ensures that alerts generated by the system are correctly displayed on the dashboard. It also checks whether security analysts can view and analyse alerts in real time. This testing phase guarantees that all system components are properly connected and functioning together to provide reliable security monitoring.

Table. 8.2: Integration Testing

Test Scenario	Input	Expected Output	Pass Criteria
Security Event Detection Workflow	Multiple failed login attempts from a user	System detects suspicious activity and generates alert	Alert appears in dashboard and stored in database
Log Monitoring Process	Server and firewall logs	Logs collected and displayed in monitoring dashboard	Logs visible in real-time dashboard

8.3 SYSTEM TESTING

System testing in the Security Information and Event Management (SIEM) system is performed to evaluate the overall functionality, performance, and usability of the fully integrated system. It ensures that all modules including log collection, log processing, data storage, event correlation, alert generation, and dashboard visualization—work together correctly according to the project requirements. This level of testing simulates real-world scenarios to verify that the system can effectively monitor and detect security events.

In this project, system testing is important to ensure that logs from client systems are correctly collected, processed, and analyzed by the SIEM server. It verifies that alerts are generated accurately and displayed properly on the dashboard for monitoring. The testing also checks how the system responds to different types of inputs, such as normal activity and suspicious behaviour.

Testers do not require deep programming knowledge, as this phase focuses on user interaction, system response, and output accuracy. It also evaluates performance factors such as processing speed, system reliability, and handling of large volumes of log data.

System testing plays a key role in identifying any remaining errors and ensures that the SIEM system is stable, efficient, and ready for deployment in a real-world environment. In addition to validating the overall functionality of the SIEM system, system testing also ensures that the system performs reliably under real-world conditions. Since the SIEM system is designed to handle continuous streams of security data, it must be capable of processing large volumes of logs without performance degradation. Therefore, system testing includes evaluating how the system behaves under both normal and peak workloads.

One of the important aspects of system testing is end-to-end workflow validation. This involves testing the complete flow of data from log generation at client systems to alert visualization on the dashboard. The testing ensures that logs are correctly collected, transmitted, processed, stored, and analysed without any data loss or delay. This comprehensive validation confirms that all system components are properly integrated and functioning as expected.

Another key focus of system testing is real-time processing capability. The SIEM system must detect and respond to threats instantly. Tests are conducted to verify that the system processes incoming logs in real time and generates alerts without delay. This is critical for identifying security incidents such as unauthorized access or brute force attacks before they cause damage.

System testing also evaluates the accuracy of threat detection. The system is tested with different types of input data, including both normal and malicious activities. This helps verify whether the system correctly identifies suspicious behaviour while avoiding false positives. For example, testing scenarios may include multiple failed login attempts, unusual access patterns, and abnormal system activities.

Another important aspect is performance testing, which measures how efficiently the system operates under different conditions. This includes testing the system's ability to handle high volumes of log data, multiple simultaneous users, and continuous monitoring. Performance testing ensures that the system remains stable and responsive even during peak usage.

System testing also includes load testing, where the system is tested with a large amount of data to evaluate its capacity. This helps determine whether the system can scale effectively as the organization grows. It also identifies potential bottlenecks that may affect system performance.

In addition, stress testing is performed to evaluate how the system behaves under extreme conditions. This involves pushing the system beyond its normal limits to observe its response. The goal is to ensure that the system can handle unexpected situations without crashing and can recover gracefully after failure.

Another critical area of system testing is security testing. Since the SIEM system deals with sensitive data, it must be protected from unauthorized access. Security testing ensures that proper authentication and authorization mechanisms are in place. It also verifies that data is securely transmitted and stored, preventing potential security breaches.

System testing also focuses on usability testing, which evaluates how easy it is for users to interact with the system. The dashboard interface is tested to ensure that it is intuitive, responsive, and user-friendly. Users should be able to easily navigate through logs, alerts, and reports without confusion.

Another important aspect is compatibility testing, which ensures that the system works correctly across different environments. The SIEM system may be deployed on different operating systems or accessed through various web browsers. Compatibility testing verifies that the system performs consistently in all these scenarios.

System testing also evaluates data integrity and consistency. It ensures that data remains accurate throughout the processing cycle. Logs collected from different sources should be stored correctly and displayed accurately on the dashboard. This is important for reliable analysis and decision-making.

Another area of testing is error handling and recovery. The system must be able to handle unexpected errors without affecting its functionality. System testing verifies that appropriate error messages are displayed and that the system can recover from failures without data loss.

System testing also includes response time analysis, which measures how quickly the system responds to user actions and generates alerts. A fast response time is essential for effective security monitoring.

In addition, the system is tested for continuous operation, ensuring that it can run 24/7 without interruptions. This is important because security monitoring must be active at all times to detect threats.

System testing also verifies the accuracy of dashboard visualization. Charts, graphs, and reports must correctly represent the underlying data. This helps users understand system behaviour and identify trends.

Another important aspect is testing alert prioritization. The system should classify alerts based on severity, allowing analysts to focus on critical issues first. Testing ensures that alerts are categorized correctly.

System testing also evaluates log retention and retrieval. The system should store logs efficiently and allow users to retrieve historical data for analysis. This is important for investigating past incidents.

In addition, integration with external systems is tested to ensure compatibility with other security tools and devices. This enhances the system's functionality and flexibility.

System testing also includes validation of configuration settings. Changes in rules or thresholds should not affect system performance negatively. Testing ensures that configurations are applied correctly.

Another important aspect is scalability testing, which verifies that the system can handle growth in data and users. This ensures long-term usability of the system.

System testing also focuses on monitoring system stability over time. Long-duration tests are conducted to ensure that the system remains stable without memory leaks or performance degradation.

Furthermore, testing includes user acceptance simulation, where real-world scenarios are replicated to evaluate system behaviour. This ensures that the system meets user expectations.

Another key aspect is testing redundancy and backup mechanisms. The system should be able to recover data in case of failure, ensuring data availability.

System testing also evaluates the effectiveness of alert notifications, ensuring that alerts reach the intended users through proper channels.

In addition, testing ensures that the system handles different types of input data, including valid, invalid, and unexpected inputs, without failure.

System testing also verifies the consistency of results across repeated operations, ensuring reliability.

Another important factor is maintainability testing, which ensures that the system can be easily updated and modified.

Finally, system testing ensures that the SIEM system is robust, reliable, scalable, and efficient, capable of handling real-world cybersecurity challenges.

Another important dimension of system testing in the SIEM system is the evaluation of data flow consistency across multiple modules. Since the system operates by transferring data between various components such as log collectors, processors, storage units, and visualization dashboards, it is essential to ensure that the data remains consistent at every stage. System testing verifies that there is no data loss, duplication, or corruption during transmission and processing. This guarantees the integrity of the system and ensures reliable security analysis.

System testing also emphasizes multi-source log validation, where logs are collected from different devices such as servers, endpoints, and network equipment. These logs may vary in format and structure, so the system must standardize them effectively. Testing ensures that logs from all sources are correctly integrated and interpreted by the SIEM system. This capability is crucial for achieving a unified view of security events across the organization.

Another key aspect is real-time alert validation under different scenarios. The system is tested with various simulated attack scenarios such as brute force attacks, unauthorized access attempts, and unusual traffic patterns. These simulations help verify whether the system can detect threats accurately and generate timely alerts. It also ensures that alerts are meaningful and provide sufficient information for further investigation.

System testing also includes network performance evaluation, which ensures that data transmission between client systems and the SIEM server is efficient and reliable. Network delays or interruptions can affect real-time monitoring, so testing is performed to verify that the system can handle such conditions without failure. This ensures uninterrupted monitoring of security events.

Another important area is concurrency testing, where multiple users or processes interact with the system simultaneously. The SIEM system must handle concurrent operations such as log ingestion, analysis, and dashboard access without conflicts. System testing verifies that the system performs efficiently under concurrent usage conditions.

System testing also focuses on data visualization accuracy. The dashboard displays various metrics, charts, and reports that help users understand system activity. Testing ensures that these visual elements accurately represent the underlying data. This is important because incorrect visualization can lead to misinterpretation of security events.

In addition, log retention policies are tested to ensure that the system stores data for the required duration. Organizations often need to retain logs for auditing and compliance purposes. System testing verifies that logs are stored securely and can be retrieved when needed.

Another significant aspect is system recovery and fault tolerance testing. The SIEM system must be able to recover from unexpected failures such as server crashes or network issues. Testing ensures that backup mechanisms are in place and that the system can resume operation without losing critical data.

System testing also includes validation of alert escalation mechanisms. In many cases, critical alerts need to be escalated to higher-level analysts or administrators. Testing ensures that the escalation process works correctly and that important alerts receive immediate attention.

Another key factor is user interaction testing, which evaluates how users interact with the system. This includes testing features such as log search, filtering, and report generation. The goal is to ensure that users can perform tasks and efficiently.

System testing also verifies integration with external security tools, such as firewalls or intrusion detection systems. This ensures that the SIEM system can work as part of a larger security infrastructure.

Another important area is testing of automated responses. Some SIEM systems can automatically take actions such as blocking IP addresses or isolating systems. Testing ensures that these automated actions are executed correctly without unintended consequences.

System testing also evaluates the efficiency of data indexing and search operations. Since the system handles large volumes of data, it must be able to quickly search and retrieve information. Testing ensures that queries return results promptly and accurately.

In addition, configuration management testing ensures that system settings can be modified without affecting performance. This includes testing changes in rules, thresholds, and system parameters.

Another important aspect is testing under different environmental conditions, such as varying network speeds or hardware configurations. This ensures that the system performs consistently across different setups.

System testing also focuses on long-term stability testing, where the system is run continuously for extended periods. This helps identify issues such as memory leaks or performance degradation over time.

Another key area is audit trail verification, which ensures that all system activities are logged and can be reviewed later. This is important for accountability and compliance purposes.

System testing also ensures that the system handles unexpected user inputs gracefully. Whether the input is incomplete, incorrect, or malicious, the system should respond appropriately without crashing.

Furthermore, testing includes evaluation of alert noise reduction mechanisms. The system should minimize unnecessary alerts and focus on significant threats. This improves efficiency and reduces workload on analysts.

System testing also verifies the effectiveness of correlation rules in detecting complex attack patterns. It ensures that multiple related events are correctly identified as a single security incident.

Another important aspect is testing of update and upgrade processes. The system should support updates without disrupting ongoing operations. Testing ensures smooth transitions during upgrades.

Finally, system testing ensures that the SIEM system meets all functional and non-functional requirements, including performance, reliability, scalability, and usability. It confirms that the system is ready for deployment and capable of handling real-world cybersecurity challenges effectively.

8.4 ACCEPTANCE TESTING

Acceptance Testing is the final stage of software testing that verifies whether the Security Information and Event Management (SIEM) system meets the organizational requirements and is ready for real-time deployment. In this project, acceptance testing ensures that the SIEM system functions correctly from the user's perspective, including monitoring, alert generation, and dashboard visualization.

This phase validates that the system behaves as expected in real-world scenarios. It ensures that security analysts can monitor logs, detect suspicious activities, and respond to alerts efficiently. It also checks whether the system provides accurate and timely alerts based on security events.

In addition to verifying that the SIEM system meets basic functional requirements, acceptance testing plays a critical role in ensuring that the system satisfies the real-world expectations of end users. Since the SIEM system is primarily used by security analysts and administrators, it must be reliable, efficient, and easy to use. Acceptance testing ensures that the system delivers a seamless user experience while maintaining high levels of accuracy and performance.

One of the primary objectives of acceptance testing is to validate the end-to-end functionality of the system. This involves testing the complete workflow, starting from log generation at client systems to alert visualization on the dashboard. The testing ensures that logs are accurately collected, processed, stored, and analysed, and that the results are correctly

presented to the user. This comprehensive validation confirms that the system is ready for deployment in a real-time environment.

Acceptance testing also focuses on user satisfaction and usability. The system should be easy to understand and operate, even for users with limited technical knowledge. The dashboard interface is evaluated to ensure that it is intuitive, responsive, and user-friendly. Users should be able to navigate through logs, filter data, and analyse alerts without difficulty. This improves the overall effectiveness of the system and increases user adoption.

Another important aspect of acceptance testing is the validation of real-time monitoring capabilities. The SIEM system must continuously monitor security events and provide instant alerts when suspicious activities are detected. Testing ensures that there are no delays in data processing or alert generation. This is crucial for preventing potential security breaches and minimizing damage.

The testing process also evaluates the accuracy of alerts generated by the system. It verifies that alerts are triggered only when specific conditions are met and that false positives are minimized. Excessive false alerts can overwhelm security analysts and reduce system efficiency. Therefore, acceptance testing ensures that alerts are meaningful, relevant, and actionable.

Acceptance testing also includes validation of different user scenarios. The system is tested with various types of inputs, including normal user behaviour and simulated attack scenarios. This helps verify that the system can handle different situations effectively. For example, scenarios such as multiple failed login attempts, unauthorized access, and unusual system activities are tested to ensure accurate detection.

Another key area of acceptance testing is performance evaluation. The system must be able to handle large volumes of log data without affecting performance. Testing ensures that the system remains responsive and stable even under heavy workloads. This is particularly important in large organizations where the volume of data is significantly higher.

Acceptance testing also verifies the reliability of the system over time. The system is tested for continuous operation to ensure that it can run without interruptions. This includes checking for issues such as memory leaks, system crashes, or performance degradation. Continuous monitoring is essential for effective security management, and the system must be capable of operating 24/7.

Another important aspect is data accuracy and integrity. Acceptance testing ensures that all data processed by the system is accurate and consistent. Logs collected from different sources should be correctly stored and displayed without any loss or corruption. This is critical for making reliable security decisions.

The testing process also evaluates the effectiveness of dashboard visualization. The system provides graphical representations of data through charts, graphs, and reports. Acceptance testing ensures that these visual elements accurately reflect the underlying data and help users understand security trends

Acceptance testing also includes validation of system configurations. The SIEM system allows customization of rules, thresholds, and settings. Testing ensures that these configurations are applied correctly and that changes do not negatively impact system performance.

Another key aspect is security validation of the system itself. Since the SIEM system handles sensitive information, it must be protected from unauthorized access. Acceptance testing verifies that authentication and authorization mechanisms are working correctly and that data is securely stored and transmitted.

Acceptance testing also evaluates the integration of the system with external components. The SIEM system may interact with other tools such as firewalls, intrusion detection systems, and network devices. Testing ensures that these integrations work seamlessly and that data is exchanged correctly.

Another important area is error handling and recovery. The system must be able to handle unexpected errors gracefully without crashing. Acceptance testing verifies that appropriate error messages are displayed and that the system can recover from failures without data loss.

The testing process also includes validation of alert prioritization and categorization. Alerts should be classified based on severity levels, allowing security analysts to focus on critical issues first. Testing ensures that this prioritization is accurate and effective.

Acceptance testing also focuses on user interaction and experience. The system should respond quickly to user actions such as searching logs, filtering data, and viewing reports. This ensures smooth interaction and improves overall usability.

Another important aspect is testing system scalability. The SIEM system should be able to handle growth in data volume and number of users. Acceptance testing verifies that the system can scale effectively without performance issues.

The system is also tested for compatibility across different environments. It should work consistently on various operating systems, browsers, and devices. This ensures flexibility and accessibility for users.

Acceptance testing also includes validation of log retention and retrieval mechanisms. The system should store logs securely and allow users to retrieve historical data. This is important for auditing and investigation purposes.

Another key area is testing automated responses. Some SIEM systems can automatically take actions such as blocking IP addresses. Acceptance testing ensures that these automated actions are performed correctly.

The testing process also evaluates the efficiency of data indexing and search operations. The system must be able to quickly retrieve information from large datasets. Acceptance testing ensures fast and accurate search functionality.

Acceptance testing also verifies the consistency of system behaviour. The system should produce the same results for the same inputs, ensuring reliability.

Another important aspect is testing backup and recovery mechanisms. The system should be able to recover data in case of failure, ensuring data availability and reliability.

Acceptance testing also includes validation of compliance requirements. The system should meet organizational and regulatory standards for data security and logging.

The testing process also evaluates the effectiveness of communication and notification systems. Alerts should be delivered to the appropriate users through proper channels.

Another key area is testing different input conditions, including valid, invalid, and unexpected inputs. The system should handle all inputs gracefully without errors.

Acceptance testing also ensures that the system supports continuous improvement and updates. New features and updates should not affect existing functionality.

Another important aspect is testing system maintainability. The system should be easy to update and manage over time.

Acceptance testing also evaluates the overall impact of the system on organizational workflow. It ensures that the system integrates smoothly into existing processes.

Another key factor is testing user roles and access control. Different users should have appropriate access levels based on their roles.

Acceptance testing also verifies the accuracy of reporting and documentation features. Reports generated by the system should be clear and useful.

Another important aspect is testing system responsiveness under different network conditions. The system should perform well even with slow or unstable connections.

Acceptance testing also includes validation of event correlation effectiveness. The system should correctly identify complex attack patterns.

Another key area is testing long-term system stability. The system should operate continuously without issues.

Sample Test Cases for Acceptance Testing:

1. When logs are generated from client systems, the SIEM system should collect and display them correctly on the dashboard.
2. When suspicious activities (e.g., multiple failed login attempts) occur, the system should generate alerts accurately.
3. If no suspicious activity is detected, the system should continue monitoring without generating false alerts.
4. If invalid or corrupted log data is received, the system should handle it without crashing and display appropriate messages.
5. When accessed from different devices or screen sizes, the dashboard should remain responsive and user-friendly.
6. When analysts monitor the system, they should be able to view logs, analyse alerts, and perform actions smoothly without interruption.

Chapter 9

SOURCE CODE

HTML CODE - LOGIN

```
<!DOCTYPE html>

<html>

<head>

    <title>SIEM Login</title>

    <link rel="stylesheet" href="/static/style.css">

</head>

<body class="login-body">

<div class="login-container">

    <div class="login-box">

        <!-- TITLE -->

        <h1>SIEM PROJECT</h1>

        <!-- FORM -->

        <form method="POST">

            <input type="text" name="username" placeholder="Username" required>

            <input type="password" name="password" placeholder="Password" required>

            <button type="submit">Login</button>

        </form>

        {% if error %}

        <p class="error">{{ error }}</p>

        {% endif %}

    </div>
```

</div>

</body>

</html>

HTML CODE – DASHBOARD

<!DOCTYPE html>

<html>

<head>

 <title>SIEM Dashboard</title>

 <link rel="stylesheet" href="/static/style.css">

</head>

<body>

<div class="container">

 <!-- SIDEBAR -->

 <div class="sidebar">

 <h2>SIEM</h2>

 <button onclick="window.location.href='/dashboard'">Home</button>

 <button onclick="window.location.href='/wazuh'">Dashboard</button>

 <button onclick="window.location.href='/logs'">Logs</button>

 <button onclick="window.location.href='/agents'">Agents</button>

 <button onclick="window.location.href='/settings'">Settings</button>

 <button onclick="window.location.href='/logout'">Logout</button>

 </div>

 <!-- CENTER -->

 <div class="center">

```
<div class="content">

    <!-- IMAGE -->

    <!-- TITLE -->

    <h1>

        DESIGN AND IMPLEMENTATION OF<br>

        SECURITY INFORMATION AND EVENT MANAGEMENT (SIEM)

    </h1>

    <!-- SUBTITLE -->

    <p>

        FOR MONITORING SECURITY EVENTS IN A BUSINESS ORGANIZATION

    </p>

</div>

</div>

</div>

</body>

</html>
```

CSS CODE

```
body {

    margin: 0;

    font-family: "Segoe UI", Arial;

    background: #f5f7fa;

}

/* MAIN LAYOUT */
```



```
.container {  
  
    display: flex;  
  
    height: 100vh;  
  
}  
  
/* SIDEBAR */  
  
.sidebar {  
  
    width: 220px;  
  
    background: #0D47A1;  
  
    color: white;  
  
    padding: 20px;  
  
}  
  
.sidebar h2 {  
  
    text-align: center;  
  
    margin-bottom: 20px;  
  
}  
  
/* SIDEBAR BUTTONS */  
  
.sidebar button {  
  
    width: 100%;  
  
    padding: 12px;  
  
    margin-top: 12px;  
  
    border: none;  
  
    background: white;  
  
    color: black;  
  
    cursor: pointer;
```

```
border-radius: 5px;

transition: 0.3s;

}

.sidebar button:hover {

    background: #e0e0e0;

    transform: scale(1.05);

}

/* CENTER AREA */

.center {

    flex: 1;

    display: flex;

    justify-content: center;

    align-items: center;

}

/* CONTENT */

.content {

    text-align: center;

    max-width: 700px;

    display: flex;

    flex-direction: column;

    align-items: center;

    animation: fadeSlide 1.5s ease-in-out;

}

/* IMAGE (SLIGHTLY BIGGER) */
```

```
.top-image {  
    width: 200px;  
    height: auto;  
    margin-bottom: 30px;  
    border-radius: 10px;  
    filter: drop-shadow(0 0 10px rgba(13,71,161,0.5));  
}
```

```
/* TITLE */
```

```
.content h1 {  
    font-size: 26px;  
    font-weight: bold;  
    color: #0D47A1;  
    line-height: 1.5;  
    margin: 10px 0;  
}
```

```
/* SUBTITLE */
```

```
.content p {  
    font-size: 16px;  
    font-weight: bold;  
    color: #0D47A1;  
    margin-top: 10px;  
}
```

```
/* ANIMATION */
```

```
@keyframes fadeSlide {
```

```
0% {
    opacity: 0;
    transform: translateY(40px);
}
100% {
    opacity: 1;
    transform: translateY(0);
}
}

/* ===== LOGIN PAGE ===== */

.login-body {
    background: #0D47A1;
    display: flex;
    justify-content: center;
    align-items: center;
    height: 100vh;
}

/* CENTER LOGIN */

.login-container {
    display: flex;
    justify-content: center;
    align-items: center;
}

/* LOGIN BOX (MATCH DASHBOARD STYLE) */
```

```
.login-box {  
  
    background: white;  
  
    padding: 40px;  
  
    width: 320px;  
  
    border-radius: 10px;  
  
    text-align: center;  
  
    /* same shadow like dashboard theme */  
  
    box-shadow: 0 0 20px rgba(13,71,161,0.2);  
  
    animation: fadeSlide 1.5s ease-in-out;  
  
}  
  
/* TITLE */  
  
.login-box h1 {  
  
    color: #0D47A1;  
  
    margin-bottom: 20px;  
  
}  
  
/* INPUT */  
  
.login-box input {  
  
    width: 100%;  
  
    padding: 12px;  
  
    margin: 10px 0;  
  
    border-radius: 5px;  
  
    border: 1px solid #ccc;  
  
}  
  
/* BUTTON */
```

```
.login-box button {  
  
    width: 100%;  
  
    padding: 12px;  
  
    background: #0D47A1;  
  
    color: white;  
  
    border: none;  
  
    border-radius: 5px;  
  
    cursor: pointer;  
  
}  
  
.login-box button:hover {  
  
    background: #08306b;  
  
}  
  
/* ERROR */  
  
.error {  
  
    color: red;  
  
    margin-top: 10px;  
  
}
```

PYTHON CODE

```
from flask import Flask, render_template, request, redirect  
  
app = Flask(__name__)  
  
# LOGIN PAGE (FIRST PAGE)  
  
@app.route('/', methods=['GET', 'POST'])  
  
def login():  
  
    if request.method == 'POST':
```

```
username = request.form.get('username')

password = request.form.get('password')

if username == 'admin' and password == 'admin':

    return redirect('/dashboard')

else:

    return render_template('login.html', error="Invalid Username or Password")

return render_template('login.html')

# Home Dashboard

@app.route('/dashboard')

def dashboard():

    return render_template('dashboard.html')

# Wazuh Dashboard

@app.route('/wazuh')

def wazuh():

    return redirect("https://192.168.56.101:5601")

# Logs Page (example)

@app.route('/logs')

def logs():

    return

redirect("https://192.168.56.101:5601/app/wazuh#/overview/?_g=(filters:!(),refreshInterval:(

pause:!t,value:0),time:(from:now-

24h,to:now))&tab=general&tabView=panels&_a=(columns:!(_source),filters:!('$state':(isIm

plicit:!t,store:appState),meta:(alias:!n,disabled:!f,index:'wazuh-alerts-

*',key:manager.name,negate:!f,params:(query:siem),removable:!f,type:phrase),query:(match:(

manager.name:(query:siem,type:phrase))))),index:'wazuh-alerts-

*',interval:auto,query:(language:kuery,query:'"),sort:!())")
```

Agents Page

@app.route('/agents')

def agents():

```
    return redirect("https://192.168.56.101:5601/app/wazuh#/agents-  
preview/?_g=(filters:!(),refreshInterval:(pause:!t,value:0),time:(from:now-  
24h,to:now))&_a=(columns:!(_source),filters:!('$state':(isImplicit:!t,store:appState),meta:(ali  
as:!n,disabled:!f,index:'wazuh-alerts-  
*',key:manager.name,negate:!f,params:(query:siem),removable:!f,type:phrase),query:(match:(  
manager.name:(query:siem,type:phrase))))),index:'wazuh-alerts-  
*',interval:auto,query:(language:kuery,query:"),sort:!())")
```

Settings Page

@app.route('/settings')

def settings():

```
    return redirect("https://192.168.56.101:5601/app/wazuh#/settings?tab=modules")
```

Logout

@app.route('/logout')

def logout():

```
    return redirect("/")
```

if __name__ == '__main__':

```
    app.run(debug=True)
```


COMMANDS EXECUTION & IMPLEMENTATION

SIEM SERVER SETUP COMMANDS

`sudo systemctl restart wazuh-indexer`

`sudo systemctl restart wazuh-manager`

`sudo systemctl restart wazuh-dashboard`

`sudo systemctl status wazuh-indexer`

`sudo systemctl status wazuh-manager`

`sudo systemctl status wazuh-dashboard`

CLIENT MACHINE SETUP

`ping 192.168.56.101`

`sudo systemctl start wazuh-agent`

`sudo systemctl status wazuh-agent`

APPLICATION EXECUTION

`python app.py`

ACCESS URLS

Wazuh Dashboard: <https://192.168.56.101:5601>

Flask UI: <http://127.0.0.1:5000>

UNAUTHORIZED COMMANDS

`sudo -k`

`sudo ls`

Chapter 10

SCREEN LAYOUTS

LOGIN PAGE

This conformation the screen indicates that authentication has been completed successfully allowing the user to proceed with utilizing the system features such as SIEM, log analysis, event collection, data analysis,

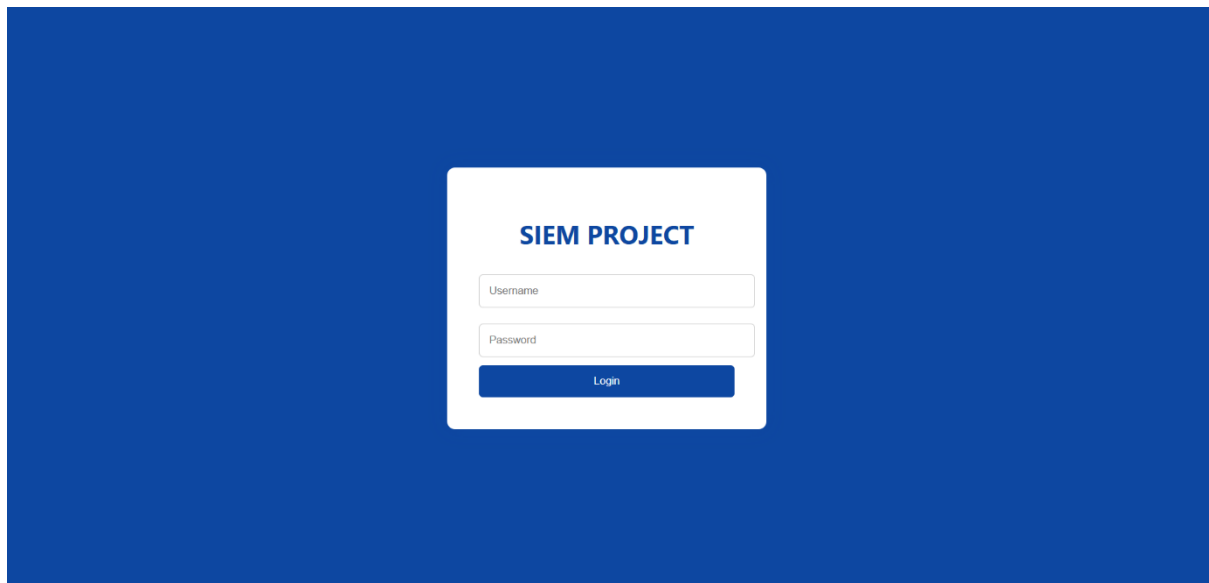


Fig. 10.1: Login Page

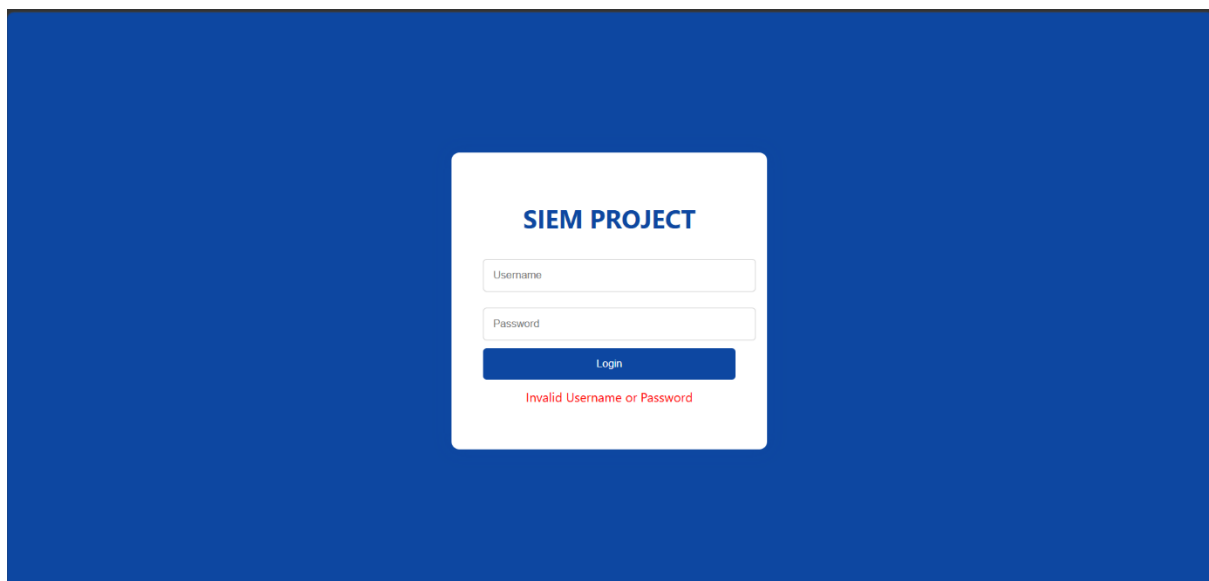


Fig. 10.2: Login Unsuccessful

Design And Implementation of Security Information and Event Management (SIEM) System for Monitoring Security Events in A Business Organization

HOME PAGE

The Home page of the SIEM System as the central dashboard for admin after successfully log.in.



Fig. 10.3: Home Page

IMPLEMENTATION

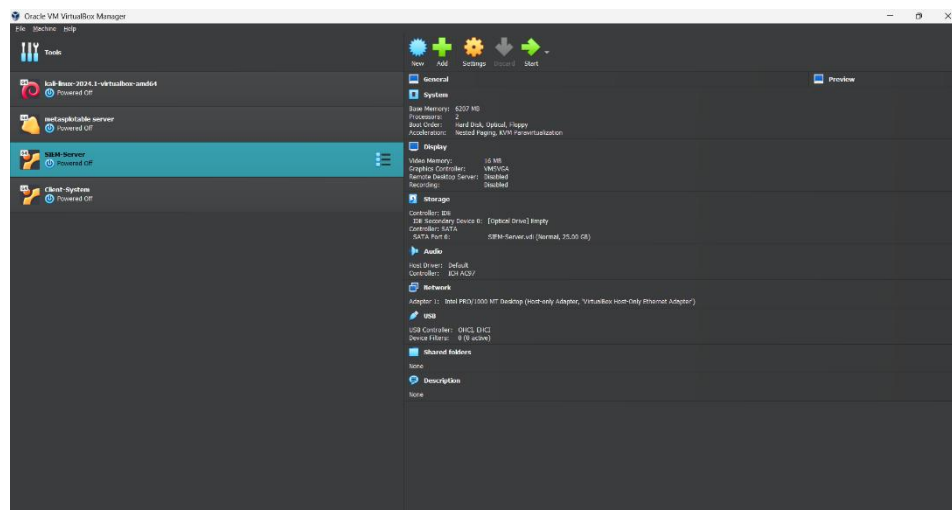


Fig. 10.4: Oracle VM VirtualBox Manager interface

This image displays the Oracle VM VirtualBox Manager interface, which is used to create and manage virtual machines for the SIEM project environment.

100

Chaitin's Lemma. $\square$

0

Design And Implementation of Security Information and Event Management (SIEM) System for Monitoring Security Events in A Business Organization

```
CGroup: /system.slice/wazuh-manager.service
├─1453 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
├─1454 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
├─1455 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
├─1456 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
├─1463 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
├─1505 /var/ossec/bin/wazuh-authd
├─1523 /var/ossec/bin/wazuh-db
├─1562 /var/ossec/bin/wazuh-execd
├─1593 /var/ossec/bin/wazuh-analysisd
├─1599 /var/ossec/bin/wazuh-syscheckd
├─1621 /var/ossec/bin/wazuh-remoted
├─1689 /var/ossec/bin/wazuh-monitord
├─1701 /var/ossec/bin/wazuh-modulesd
└─2542 find /var/log -type f -ls

Mar 11 15:20:29 siem env[1699]: 2026/03/11 15:20:29 wazuh-modulesd: WARNING: (1230): Invalid element in the configuration: 'provider'.
Mar 11 15:20:29 siem env[1699]: 2026/03/11 15:20:29 wazuh-modulesd: WARNING: (1230): Invalid element in the configuration: 'provider'.
Mar 11 15:20:29 siem env[1699]: 2026/03/11 15:20:29 wazuh-modulesd: WARNING: (1230): Invalid element in the configuration: 'provider'.
Mar 11 15:20:29 siem env[1699]: 2026/03/11 15:20:29 wazuh-modulesd: WARNING: (1230): Invalid element in the configuration: 'provider'.
Mar 11 15:20:29 siem env[1699]: 2026/03/11 15:20:29 wazuh-modulesd:router: INFO: Loaded router module.
Mar 11 15:20:29 siem env[1699]: 2026/03/11 15:20:29 wazuh-modulesd:content_manager: INFO: Loaded content_manager module.
Mar 11 15:20:29 siem env[1699]: 2026/03/11 15:20:29 wazuh-modulesd:inventory-harvester: INFO: Loaded inventory harvester module.
Mar 11 15:20:30 siem env[1387]: Started wazuh-modulesd...
Mar 11 15:20:32 siem env[1387]: Completed.
Mar 11 15:20:32 siem systemd[1]: Started wazuh-manager.service - Wazuh manager.
chaitu@siem:~$ C
chaitu@siem:~$ sudo systemctl status wazuh-dashboard
wazuh-dashboard.service - wazuh-dashboard
Loaded: loaded (/etc/systemd/system/wazuh-dashboard.service; enabled; preset: enabled)
Active: active (running) since Wed 2026-03-11 15:17:07 UTC; 3min 57s ago
Main PID: 737 (node)
Tasks: 11 (limit: 7886)
Memory: 274.0M (peak: 356.2M)
CPU: 24.485s
CGroup: /system.slice/wazuh-dashboard.service
└─737 /usr/share/wazuh-dashboard/node/bin/node --no-warnings --max-http-header-size=65536 --unhandled-rejections=warn /usr/share/wazuh-dashboard/s
Mar 11 15:19:11 siem opensearch-dashboards[737]: {"type":"log","@timestamp":"2026-03-11T15:19:11Z","tags":["error","opensearch","data"],"pid":737,"message":"[E
Mar 11 15:19:14 siem opensearch-dashboards[737]: {"type":"log","@timestamp":"2026-03-11T15:19:14Z","tags":["error","opensearch","data"],"pid":737,"message":"[E
Mar 11 15:19:16 siem opensearch-dashboards[737]: {"type":"log","@timestamp":"2026-03-11T15:19:16Z","tags":["error","opensearch","data"],"pid":737,"message":"[E
Mar 11 15:19:19 siem opensearch-dashboards[737]: {"type":"log","@timestamp":"2026-03-11T15:19:19Z","tags":["error","opensearch","data"],"pid":737,"message":"[E
Mar 11 15:19:22 siem opensearch-dashboards[737]: {"type":"log","@timestamp":"2026-03-11T15:19:22Z","tags":["info","savedobjects-service"],"pid":737,"message":"[I
Mar 11 15:19:22 siem opensearch-dashboards[737]: {"type":"log","@timestamp":"2026-03-11T15:19:22Z","tags":["info","plugins-system"],"pid":737,"message":"Starti
Mar 11 15:19:25 siem opensearch-dashboards[737]: {"type":"log","@timestamp":"2026-03-11T15:19:25Z","tags":["listening","info"],"pid":737,"message":"Server runn
Mar 11 15:19:25 siem opensearch-dashboards[737]: {"type":"log","@timestamp":"2026-03-11T15:19:25Z","tags":["info","http","server","OpenSearchDashboards"],"pid":7
Mar 11 15:20:00 siem opensearch-dashboards[737]: {"type":"log","@timestamp":"2026-03-11T15:20:00Z","tags":["error","plugins","wazuh","cron-scheduler"],"pid":73
Mar 11 15:20:00 siem opensearch-dashboards[737]: {"type":"log","@timestamp":"2026-03-11T15:20:00Z","tags":["error","plugins","wazuh","cron-scheduler"],"pid":73
chaitu@siem:~$
```

Fig. 10.9: Wazuh Dashboard Service Status

This image shows the terminal output of the command `sudo systemctl status wazuh-dashboard` executed on the SIEM server. The output confirms that the Wazuh Dashboard service is active and running, as indicated by the status “*active (running)*”.

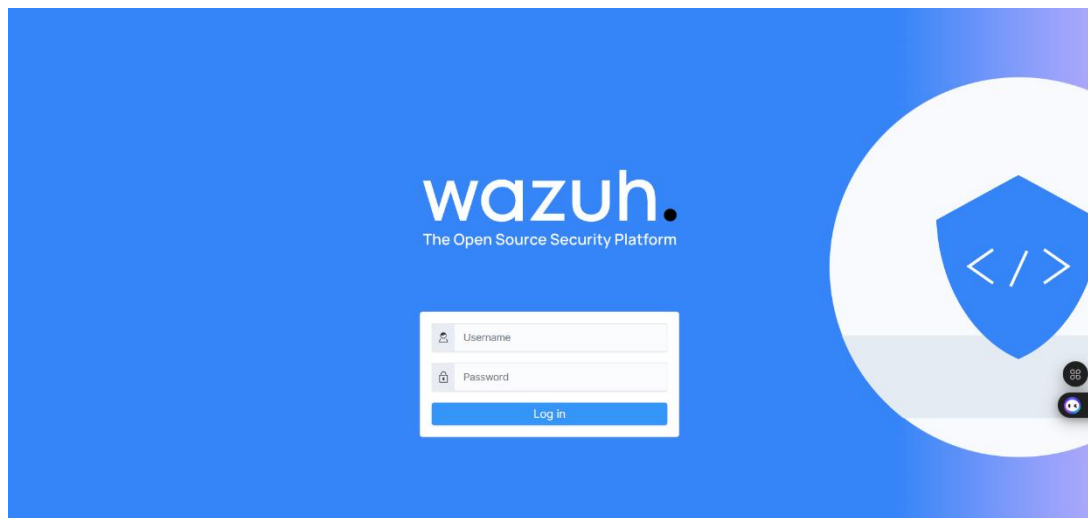


Fig. 10.10: Wazuh Dashboard Login Interface

This image shows the Wazuh web dashboard login page, which serves as the user interface for accessing the SIEM system. The screen displays the Wazuh logo along with the title “*The Open Source Security Platform*”. It includes input fields for username and password, along with a login button for authentication.

Design And Implementation of Security Information and Event Management (SIEM) System for Monitoring Security Events in A Business Organization

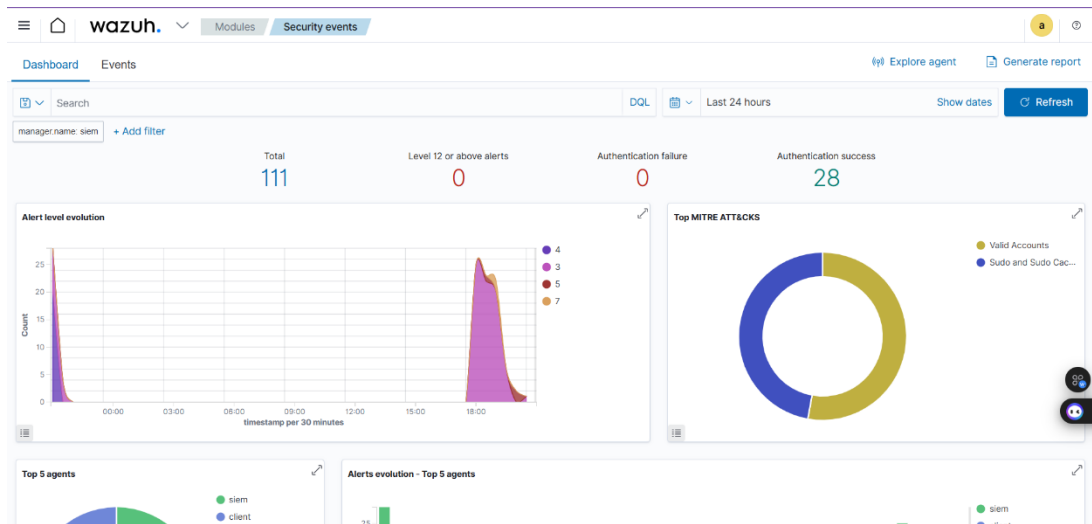


Fig. 10.11: Wazuh Security Events Dashboard

This image shows the Wazuh Security Events Dashboard, which is the graphical interface used for monitoring and analysing security activities within the SIEM system. The dashboard provides a real-time overview of security events collected from different systems and agents.

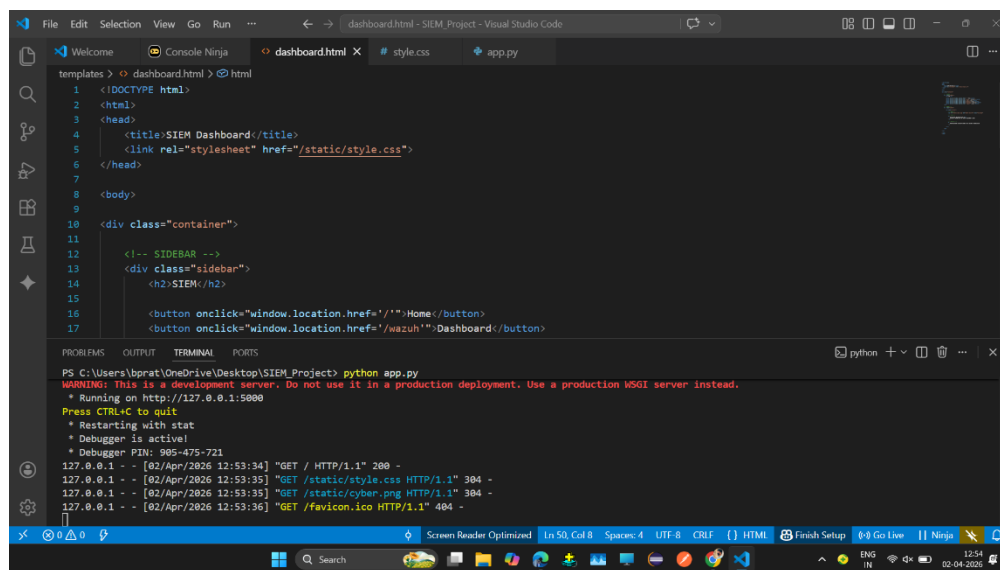


Fig. 10.12: SIEM Web Application Development

This image shows the development of a custom SIEM dashboard web application using Visual Studio Code (VS Code). The file dashboard.html is open in the editor, which contains the structure of the web interface built using HTML. The code includes elements such as the page title, stylesheet linking (style.css), container layout, sidebar, and navigation buttons for Home and Dashboard sections.

Design And Implementation of Security Information and Event Management (SIEM) System for Monitoring Security Events in A Business Organization

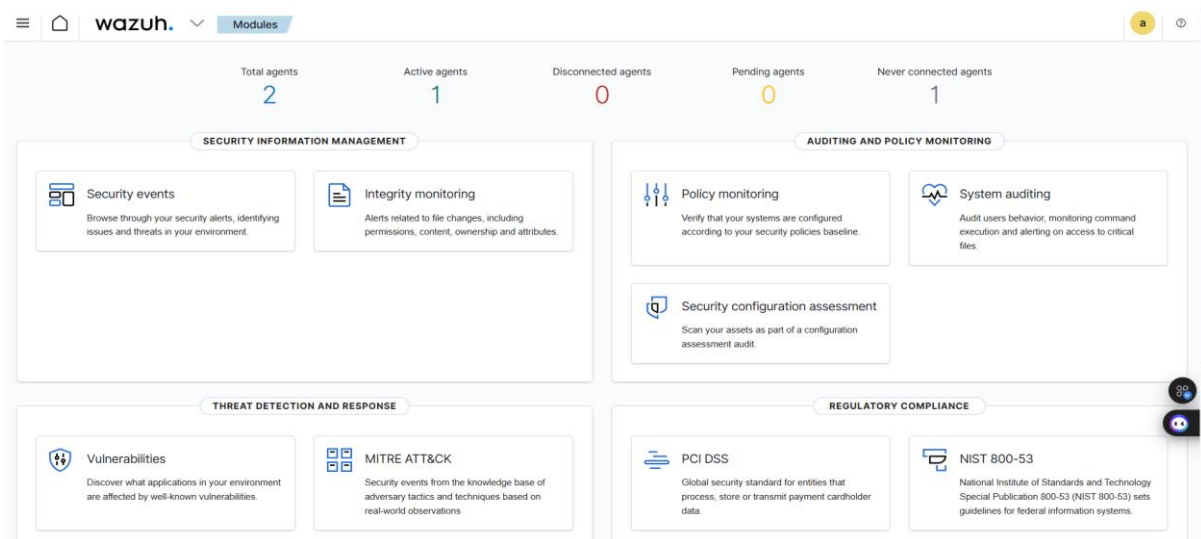
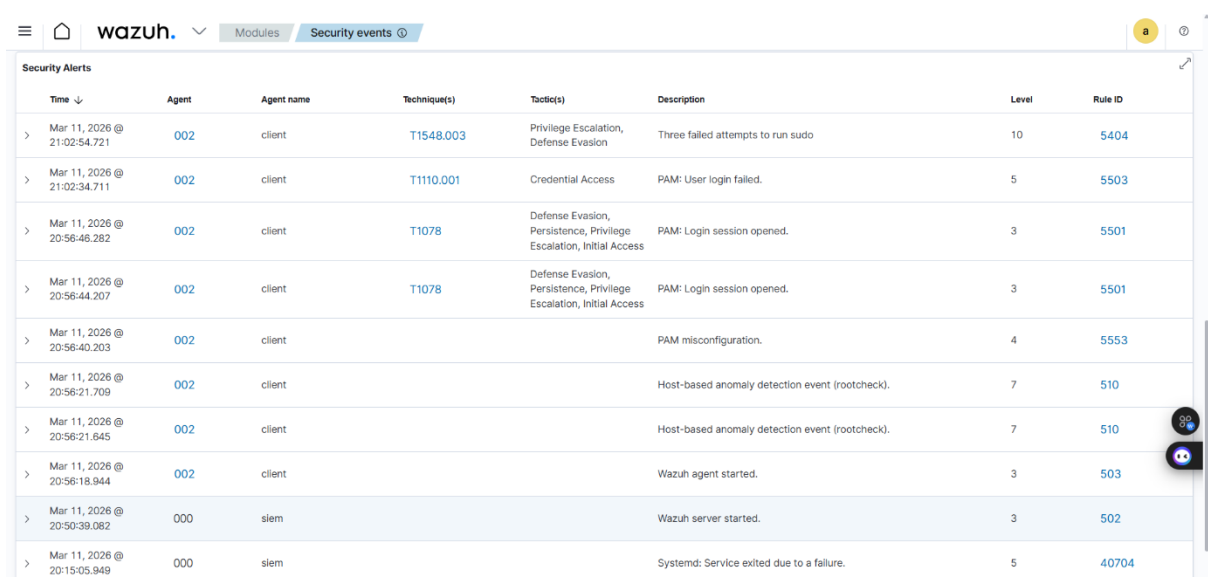


Fig. 10.13: Wazuh Modules Dashboard Overview

This image displays the Wazuh Modules Dashboard, which provides a structured overview of different security functionalities available within the SIEM system. It shows how various modules are organized to support comprehensive security monitoring and management.



The dashboard displays a table of security alerts generated in real-time based on system activities and detected events. The table includes columns for Time, Agent, Agent name, Technique(s), Tactics, Description, Level, and Rule ID.

Time	Agent	Agent name	Technique(s)	Tactics	Description	Level	Rule ID
Mar 11, 2026 @ 21:02:54.721	002	client	T1548.003	Privilege Escalation, Defense Evasion	Three failed attempts to run sudo	10	5404
Mar 11, 2026 @ 21:02:34.711	002	client	T1110.001	Credential Access	PAM: User login failed.	5	5503
Mar 11, 2026 @ 20:56:46.282	002	client	T1078	Defense Evasion, Persistence, Privilege Escalation, Initial Access	PAM: Login session opened.	3	5501
Mar 11, 2026 @ 20:56:44.207	002	client	T1078	Defense Evasion, Persistence, Privilege Escalation, Initial Access	PAM: Login session opened.	3	5501
Mar 11, 2026 @ 20:56:40.203	002	client			PAM misconfiguration.	4	5553
Mar 11, 2026 @ 20:56:21.709	002	client			Host-based anomaly detection event (rootcheck).	7	510
Mar 11, 2026 @ 20:56:21.645	002	client			Host-based anomaly detection event (rootcheck).	7	510
Mar 11, 2026 @ 20:56:18.944	002	client			Wazuh agent started.	3	503
Mar 11, 2026 @ 20:50:39.082	000	siem			Wazuh server started.	3	502
Mar 11, 2026 @ 20:15:05.949	000	siem			Systemd: Service exited due to a failure.	5	40704

Fig. 10.14: Wazuh Security Alerts – Final Output Dashboard

This image represents the final output of the SIEM system, displayed through the Wazuh Security Events interface. It shows a detailed table of security alerts generated in real-time based on system activities and detected events.

Chapter 11

FUTURE ENHANCEMENTS

The development and implementation of the Security Information and Event Management (SIEM) system in this project successfully demonstrate how modern cybersecurity challenges can be addressed through centralized monitoring and intelligent analysis of security events. In today's digital era, organizations are increasingly dependent on IT infrastructure, which exposes them to various cyber threats such as unauthorized access, malware attacks, insider threats, and data breaches. Traditional security systems, which operate independently, are no longer sufficient to handle these complex and evolving threats. This project highlights the importance of integrating security tools into a unified platform to improve overall security management.

The proposed SIEM system provides a centralized solution for collecting, processing, and analysing logs from multiple sources such as client systems, servers, and network devices. By consolidating logs into a single platform, the system eliminates the need for manual analysis of scattered data. This significantly reduces complexity and enhances the efficiency of security operations. The ability to view all security events in one place improves visibility and allows security analysts to monitor system activities more effectively.

One of the key achievements of this project is the implementation of real-time monitoring and event correlation. The system continuously analyses incoming logs and correlates events to identify suspicious patterns. This enables the detection of complex attacks such as brute force attempts, privilege escalation, and unauthorized access. Real-time alert generation ensures that security teams are immediately notified of potential threats, allowing them to respond quickly and minimize damage.

The integration of Wazuh and ELK Stack (Elasticsearch, Logstash, Kibana) plays a crucial role in the system's functionality. These open-source tools provide powerful capabilities for log management, indexing, visualization, and threat detection. The Wazuh Manager analyses logs and generates alerts, while the Wazuh Dashboard provides a user-friendly interface for monitoring and analysis. The use of open-source technologies makes the system cost-effective and suitable for both academic and organizational use.

Another important outcome of the project is the successful creation of a virtual lab environment using tools like VirtualBox. This environment simulates a real-world network setup, including multiple systems such as SIEM server, client machines, and attack systems. It allows safe testing of security scenarios without affecting real systems. The ability to simulate attacks and observe system responses enhances the understanding of cybersecurity concepts and practical implementation.

The system also demonstrates strong capabilities in threat detection and analysis. By using predefined rules and integrating MITRE ATT&CK techniques, the system provides detailed insights into different types of attacks. This helps security analysts understand attacker behaviour and take appropriate actions. The classification of alerts based on severity levels further improves incident prioritization and response efficiency.

In addition to detection, the project emphasizes effective visualization and reporting. The dashboard provides graphical representations of security events, including charts, graphs, and metrics. These visualizations make it easier to identify trends, analyse patterns, and understand system behaviour. The ability to generate reports supports decision-making and helps organizations improve their security strategies.

The project also highlights the importance of testing in software development, including unit testing, integration testing, system testing, and acceptance testing. Each testing phase ensures that the system functions correctly, performs efficiently, and meets user requirements. Thorough testing improves system reliability and ensures that the SIEM solution is ready for real-world deployment.

Another significant advantage of the proposed system is its scalability and flexibility. The system can be easily expanded by adding more log sources, rules, and configurations. This makes it suitable for organizations of different sizes and allows it to adapt to changing requirements. The modular design ensures that new features can be added without affecting existing functionality.

The project also contributes to improving incident response capabilities. By providing detailed information about security events, including timestamps, source details, and attack techniques, the system enables faster and more accurate investigation of incidents. This reduces response time and helps prevent further damage.

Furthermore, the SIEM system enhances organizational security posture by providing continuous monitoring and proactive threat detection. Instead of reacting to incidents after they occur, the system helps identify potential risks in advance. This proactive approach is essential in modern cybersecurity, where threats are constantly evolving.

Despite its advantages, the project also highlights some limitations. The system relies on predefined rules, which may not detect completely new or unknown attack patterns. Additionally, handling very large volumes of data may require more advanced infrastructure and optimization techniques. These limitations can be addressed in future work by incorporating machine learning and advanced analytics for improved threat detection.

In conclusion, the SIEM system developed in this project provides a comprehensive, efficient, and cost-effective solution for security monitoring and management. It successfully overcomes the limitations of traditional systems by offering centralized log management, real-time analysis, event correlation, and effective visualization. The project demonstrates how modern cybersecurity tools can be integrated to build a robust security solution.

Overall, this project not only enhances technical knowledge but also provides practical insights into real-world cybersecurity implementation. It serves as a strong foundation for further research and development in the field of security analytics and threat intelligence. The SIEM system proves to be an essential tool for organizations aiming to strengthen their security infrastructure and protect their digital assets from potential threats.

The Security Information and Event Management (SIEM) system developed in this project provides a strong foundation for centralized security monitoring and threat detection. However, with the rapid evolution of cybersecurity threats and technologies, there is significant scope for further enhancement and improvement. Future developments can focus on increasing system intelligence, scalability, automation, and integration with advanced technologies to make the SIEM system more powerful and efficient.

One of the most important future enhancements is the integration of Machine Learning (ML) and Artificial Intelligence (AI) techniques into the SIEM system. Currently, the system primarily relies on predefined rules and patterns to detect threats. While effective, this approach may not detect unknown or zero-day attacks. By incorporating machine learning algorithms, the system can analyse historical data, learn from patterns, and automatically detect anomalies and unusual behaviour. This will significantly improve threat detection accuracy and enable the system to identify advanced and previously unseen cyber-attacks.

Another major enhancement is the implementation of automated incident response mechanisms. In the current system, alerts are generated for security analysts to review and take action. Future versions of the SIEM system can include automated responses such as blocking suspicious IP addresses, disabling compromised user accounts, or isolating affected systems. This will reduce response time and minimize the impact of security incidents.

The system can also be enhanced by improving scalability and performance optimization. As organizations grow, the volume of log data increases significantly. Future improvements can include the use of distributed architectures, cloud-based storage, and big data technologies such as Apache Kafka and Hadoop. These technologies will enable the system to handle large-scale data efficiently and maintain real-time performance.

Another important enhancement is the integration of cloud security monitoring. With the increasing adoption of cloud platforms such as AWS, Azure, and Google Cloud, organizations require monitoring solutions that can handle cloud environments. Future SIEM systems can be extended to collect and analyse logs from cloud services, ensuring comprehensive security coverage across both on-premises and cloud infrastructures.

The system can also be improved by incorporating advanced threat intelligence feeds. By integrating external threat intelligence sources, the SIEM system can stay updated with the

latest cyber threats, vulnerabilities, and attack patterns. This will enhance the system's ability to detect and respond to emerging threats.

Another area of enhancement is user behaviour analytics (UBA). By analysing user activities and behaviour patterns, the system can detect insider threats and suspicious user actions. For example, unusual login times, access to sensitive data, or abnormal system usage can be identified and flagged as potential threats.

The future SIEM system can also include improved visualization and reporting features. While the current dashboard provides basic insights, advanced visualization tools such as interactive dashboards, predictive analytics, and customizable reports can be implemented. This will help users better understand security trends and make informed decisions.

Another enhancement is the development of a mobile-based monitoring application. This will allow security analysts to monitor alerts and system activities remotely using smartphones or tablets. Real-time notifications can be sent to users, enabling faster response to critical incidents.

The system can also be enhanced by implementing role-based access control (RBAC). This feature ensures that users have access only to the information and functionalities relevant to their roles. It improves security and prevents unauthorized access to sensitive data.

Another important future enhancement is the integration of blockchain technology for log integrity. Logs are critical for security analysis, and ensuring their integrity is essential. By using blockchain, logs can be stored in a tamper-proof manner, ensuring that they cannot be altered or deleted.

The system can also benefit from improved alert management and prioritization. Advanced algorithms can be used to reduce false positives and prioritize alerts based on risk levels. This will help security analysts focus on critical threats and improve overall efficiency.

Another area of improvement is the implementation of multi-language support and enhanced user interface design. This will make the system more user-friendly and accessible to a wider range of users.

The SIEM system can also be extended to support Internet of Things (IoT) security monitoring. As IoT devices become more common, they introduce new security challenges. Future

enhancements can include monitoring logs and activities from IoT devices to ensure their security.

Another enhancement is the addition of compliance automation features. The system can automatically generate compliance reports based on standards such as GDPR, ISO 27001, and HIPAA. This will help organizations meet regulatory requirements more efficiently.

The system can also incorporate predictive analytics, which can forecast potential security threats based on historical data. This proactive approach will help organizations prevent attacks before they occur.

Another important improvement is the implementation of data encryption and advanced security mechanisms to protect sensitive information within the SIEM system. This ensures confidentiality and prevents unauthorized access.

The future system can also include integration with DevSecOps practices, allowing security to be incorporated into the software development lifecycle. This will help organizations build secure applications from the beginning.

Additionally, the system can be enhanced by providing training modules and simulation environments for security analysts. This will help users understand how to respond to different types of cyber threats effectively.

Another enhancement is the implementation of high availability and fault tolerance mechanisms. This ensures that the system remains operational even in case of hardware or software failures.

Finally, continuous improvement can be achieved by incorporating feedback mechanisms where users can report issues and suggest improvements. This will help refine the system and make it more effective over time.

Chapter 12

BIBLIOGRAPHY

- [1] Behl, A., Behl, K., & Behl, K. (2023). Cybersecurity and Cyberwar: What Everyone Needs to Know. Oxford University Press.
- [2] Scarfone, K., & Mell, P. (2022). Guide to Intrusion Detection and Prevention Systems (IDPS). National Institute of Standards and Technology (NIST Special Publication 800-94).
- [3] Behl, A., & Behl, K. (2021). Security Information and Event Management (SIEM) Implementation. CRC Press.
- [4] Wazuh Inc. (2024). Wazuh Documentation: Open Source Security Platform. Retrieved from <https://documentation.wazuh.com>
- [5] Elastic N.V. (2024). Elastic Stack (ELK) Documentation: Elasticsearch, Logstash, Kibana. Retrieved from <https://www.elastic.co/guide>
- [6] Kent, K., & Souppaya, M. (2022). Guide to Computer Security Log Management. NIST Special Publication 800-92.
- [7] Behl, A. (2023). Cybersecurity Analytics: Techniques and Tools for Security Monitoring. Springer.
- [8] MITRE Corporation. (2024). MITRE ATT&CK Framework. Retrieved from <https://attack.mitre.org>
- [9] Rouse, M. (2023). What is SIEM (Security Information and Event Management)?. TechTarget.
- [10] Stallings, W., & Brown, L. (2022). Computer Security: Principles and Practice. Pearson Education.
- [11] NIST. (2023). Cybersecurity Framework (CSF). National Institute of Standards and Technology.
- [12] SANS Institute. (2022). Security Monitoring and SIEM Best Practices.
- [13] Cisco Systems. (2023). Introduction to Cybersecurity and Network Security Concepts.
- [14] Kizza, J. M. (2021). Guide to Computer Network Security. Springer.

- [15] Bace, R., & Mell, P. (2022). Intrusion Detection Systems. NIST Publications.
- [16] Splunk Inc. (2024). SIEM and Log Management Concepts. Retrieved from <https://www.splunk.com>
- [17] IBM Security. (2023). What is SIEM? Security Information and Event Management Explained.
- [18] ENISA. (2022). Cybersecurity Threat Landscape Report. European Union Agency for Cybersecurity.
- [19] Cloud Security Alliance. (2023). Security Guidance for Critical Areas of Focus in Cloud Computing.
- [20] Google Cloud. (2024). Security Operations and SIEM Solutions.